

Deep learning
Master's Degree in Artificial Intelligence
2025/26

Unit 5B - Autoencoders

Index

- 5B.1 Introduction
- 5B.2 Undercomplete autencoders
- 5B.3 Sparse autoencoders
- 5B.4 Applying autoencoders for anomaly detection
- 5B.5 Denoising autoencoders
- 5B.6 Variational autoencoders
- 5B.7 Implementation of a variational autoencoder
- 5B.8 Using AEs to generate new faces
- 5B.9 Using VAEs to generate new faces

5B.1 Introduction

Autoencoders are neural networks that:

- **learn latent representations** or codings: dense representations from the data with lower dimensionality
- are capable of **unsupervised training** (training sets are unlabeled)
- act as **feature detectors** (useful for unsupervised pretraining)

Analogies:

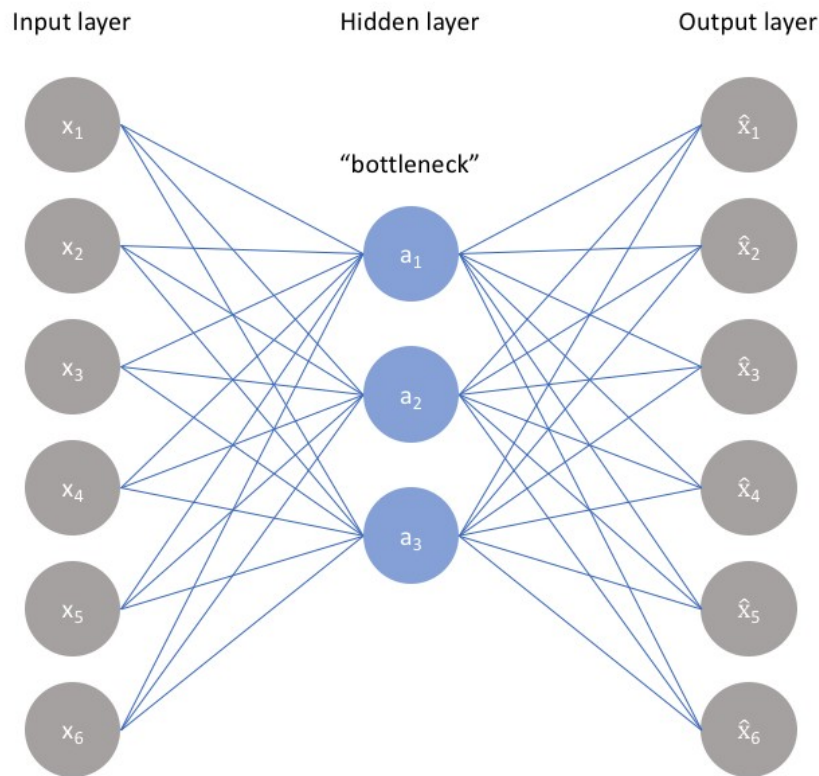
- Expert chess players are capable of memorize the positions of all pieces in the board just looking at it for five seconds. Only **when pieces are in realistic positions**.
- MP3 reduces the dimensionality of the audio in the encoding process, and recreates the original audio in the decoding process

In both cases, the unnecessary information is removed



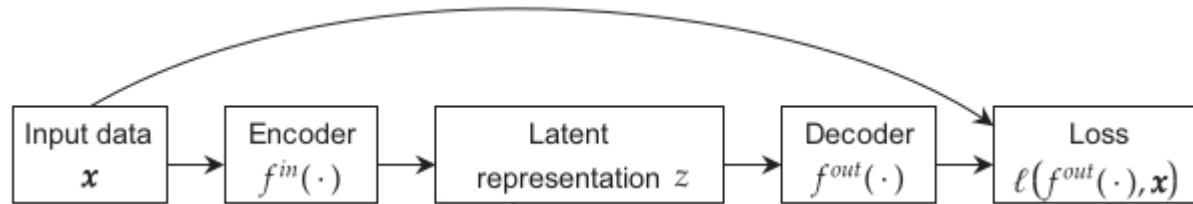
An autoencoder is composed by three parts:

- **An encoder** (*recognition network*): converts the input to a latent representation. It compresses the information into an encoded representation much smaller than the input data.
- The **bottleneck**: a module that contains the compressed representation.
- **A decoder** (*generative network*): converts the internal representation to an output. It reconstructs the data back from its encoded form.



The encoder (decoder) reduces (increases) the dimensionality of the input **in steps** before reaching the bottleneck

- Encoding transforms the input x into a new representation z
- z can be useful in clustering, search, classification....
- Decoding tries to recover the original input x



This assumes that there is a representation z to be found that captures information about x

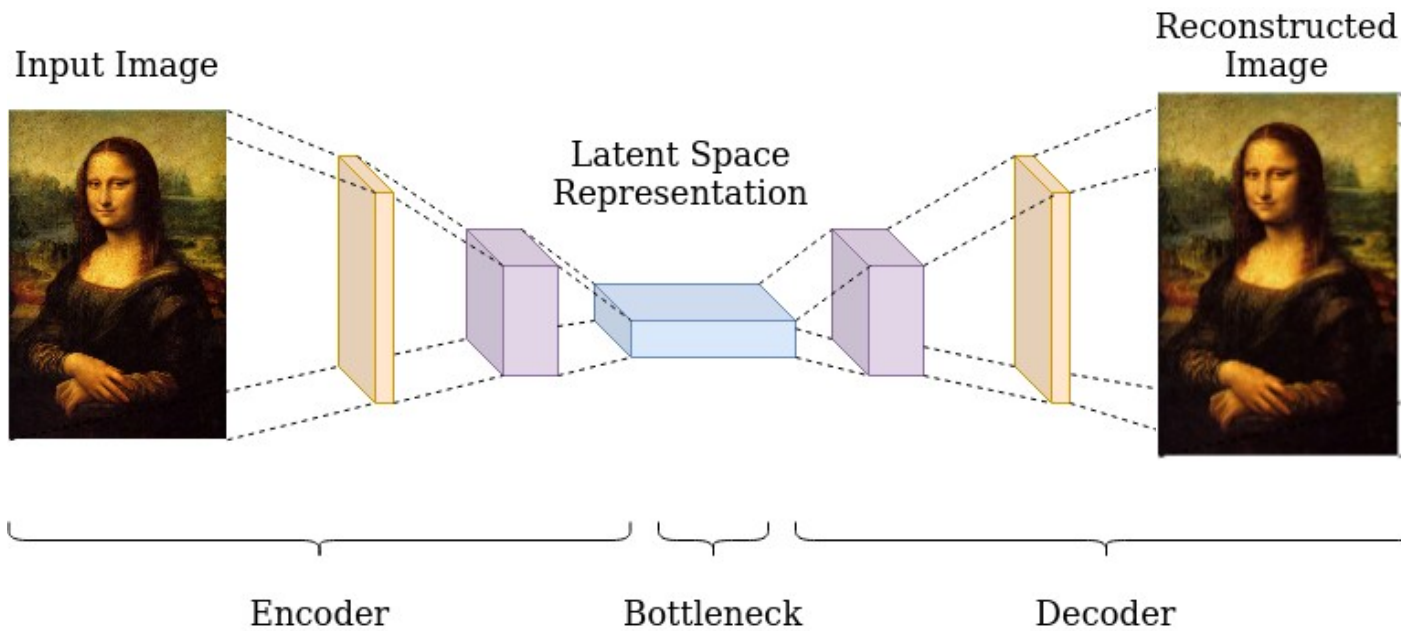
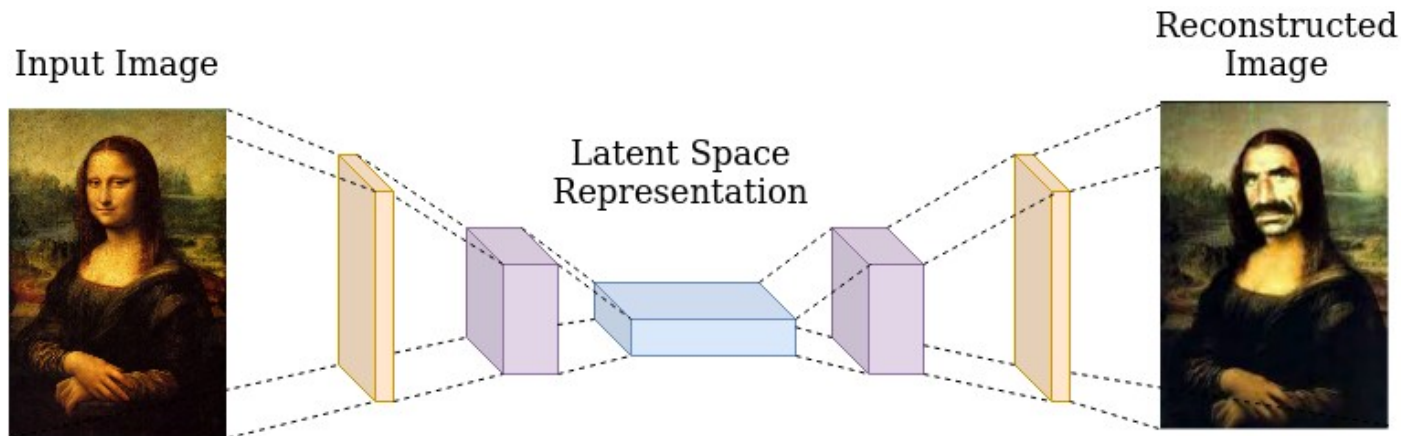
The representation z is unknown: it emerges from the training

z is the **latent representation**

Dimensionality of the latent representation?

- Too big: captures not relevant information
- Too small: little information extracted from the input, then the bigger the chances of discarding important information.

Autoencoders are trained to reduce the dimension of samples to the bottleneck dimension, and reconstruct the samples as faithfully as possible



Autoencoders vs. PCA (Principal Component Analysis)

Principal component analysis (PCA) is a popular technique for analyzing large datasets containing a high number of dimensions/features per observation, increasing the interpretability of data while preserving the maximum amount of information, and enabling the visualization of multidimensional data.



Alter the weights W so that the difference between the original data and the decoded version of the encoded data is as small as possible (and please make sure the encoder is capturing as much variance in the data as possible).

← This is the optimization problem that PCA tries to solve

$$\begin{array}{l} \text{minimize}_W \left\| X - XW W^T \right\|_2^2 \\ \text{subject to } W^T W = I \end{array}$$

We can rewrite the equation using terminology from NNs

Alter the weights W so that the difference between the original data and the decoded version of the encoded data is as small as possible (and please make sure the encoder is capturing as much variance in the data as possible).

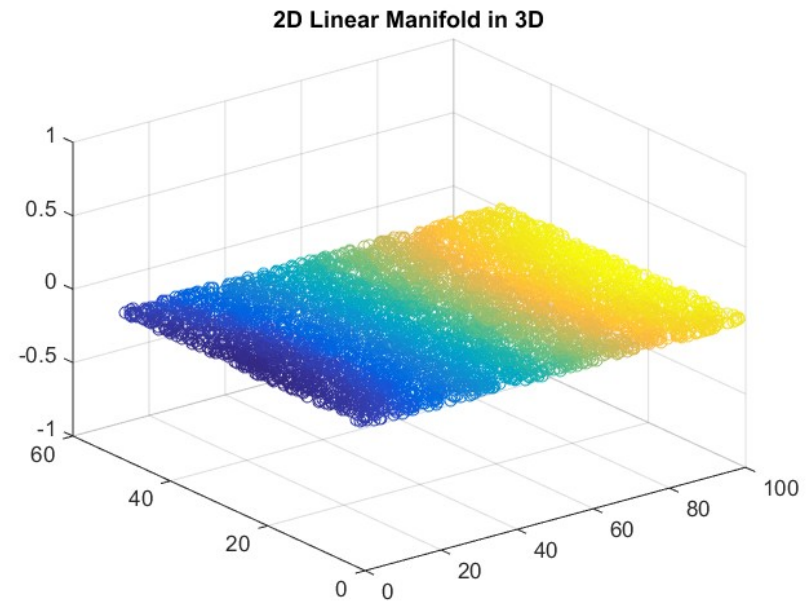
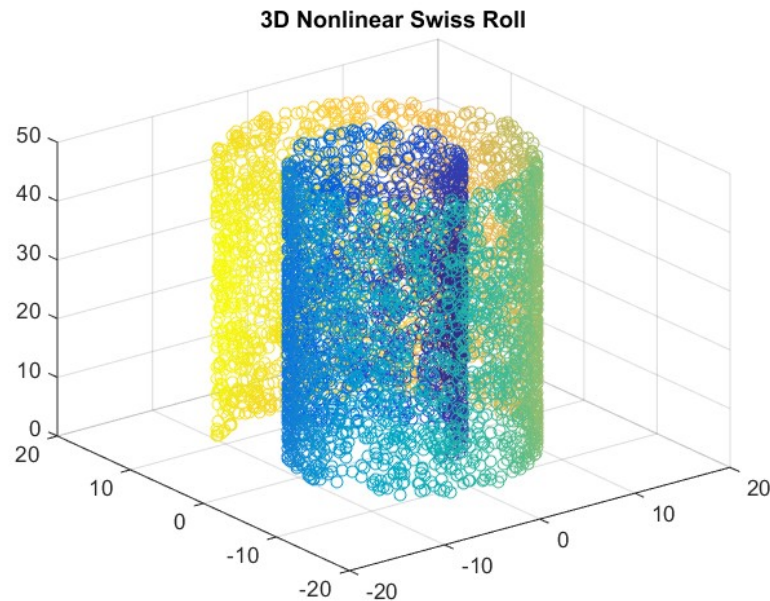
$$\|W^T W - I\|_2^2 + \sum_{i=1}^n \ell \left(\underbrace{f(x_i)}_{f^{out}(f^{in}(x_i))}, \underbrace{x_i}_{y_i} \right)$$

A single layered autoencoder with a linear activation function is equivalent to PCA.

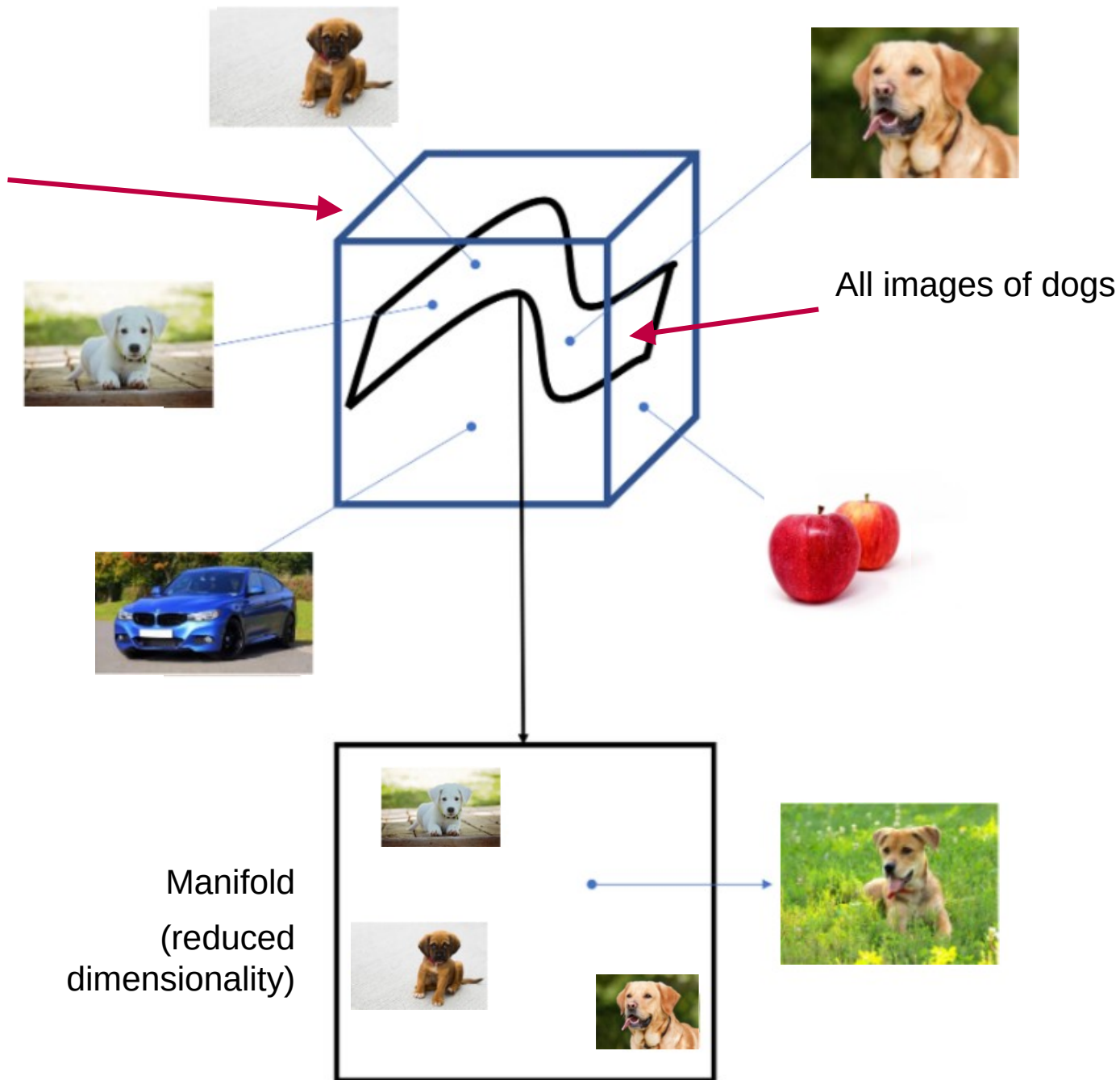
Autoencoders vs. PCA (Principal Component Analysis)

In mathematics, a **manifold** is a topological space that locally resembles Euclidean space near each point. One-dimensional manifolds are lines and circles, higher dimensional manifolds are surfaces (planes, spheres, torus, etc.)

Autoencoders are capable of learning non-linear manifolds



Vector space of all the possible images



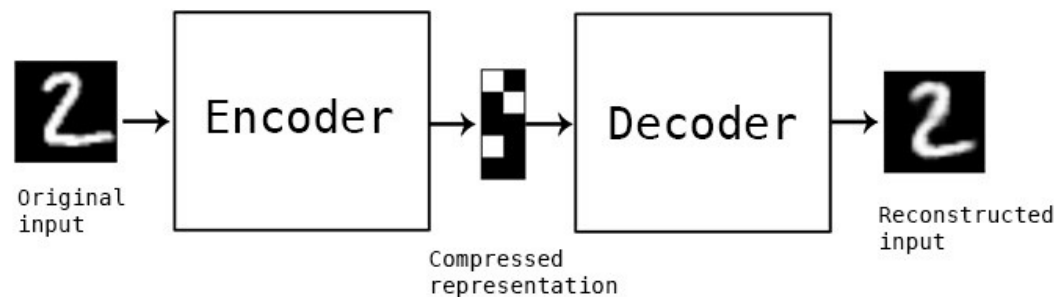
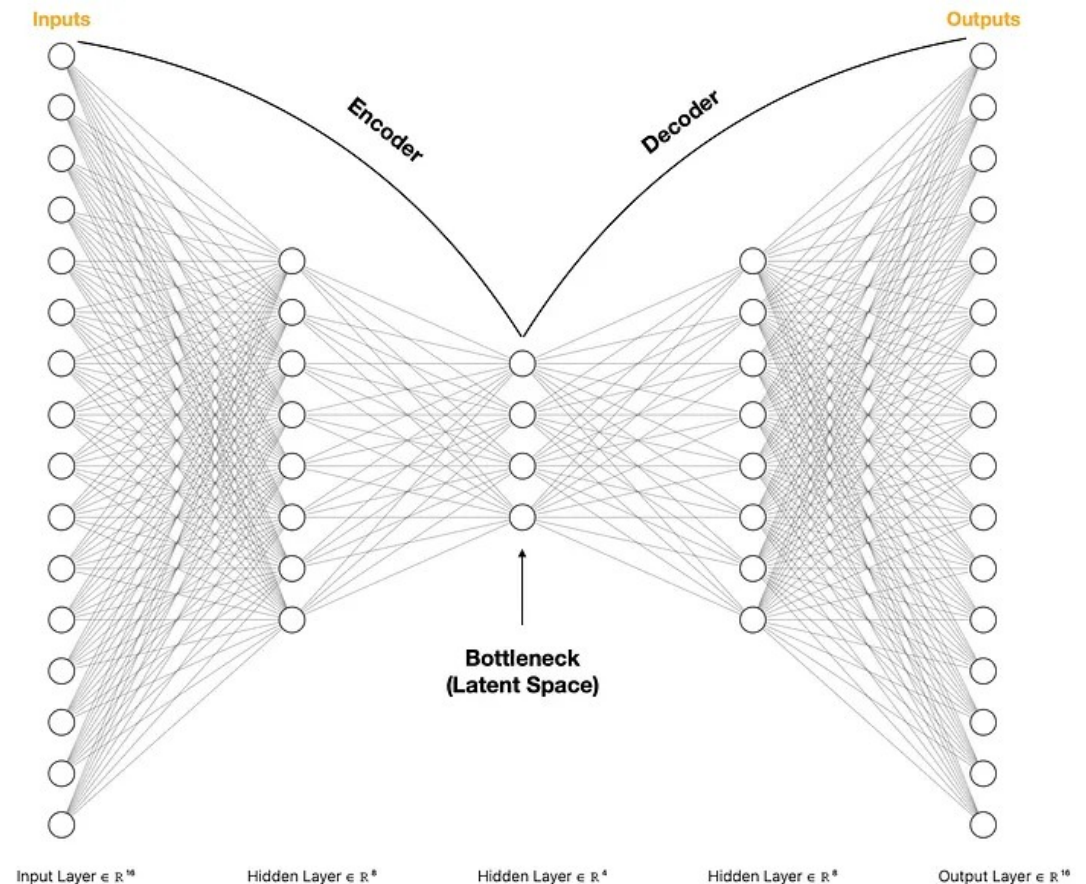
5B.2 Undercomplete autoencoders

Undercomplete autoencoders are the simplest way to construct an autoencoder.

Main idea: limit the number of nodes in the bottleneck (limit the information that can flow through the network)

There is no regularization term: we train the model according to the **reconstruction loss**.

Since there is no regularization term, **the only way to ensure that the model is not just memorizing the data is to reduce the size of the bottleneck.**



```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
encoder=Sequential()
encoder.add(Dense(256, activation='relu',input_shape=(784,)))
encoder.add(Dense(48, activation='relu'))
encoder.add(Dense(2))
encoder.build()

```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 256)	200,960
dense_1 (Dense)	(None, 48)	12,336
dense_2 (Dense)	(None, 2)	98

Total params: 213,394 (833.57 KB)

Trainable params: 213,394 (833.57 KB)

Encoder and decoder are symmetrical

```

decoder=Sequential()
decoder.add(Dense(48, input_shape=(2,)))
decoder.add(Dense(256, activation='relu'))
decoder.add(Dense(784, activation='sigmoid'))
decoder.build()
decoder.summary()

```

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 48)	144
dense_4 (Dense)	(None, 256)	12,544
dense_5 (Dense)	(None, 784)	201,488

Total params: 214,176 (836.62 KB)

Trainable params: 214,176 (836.62 KB)

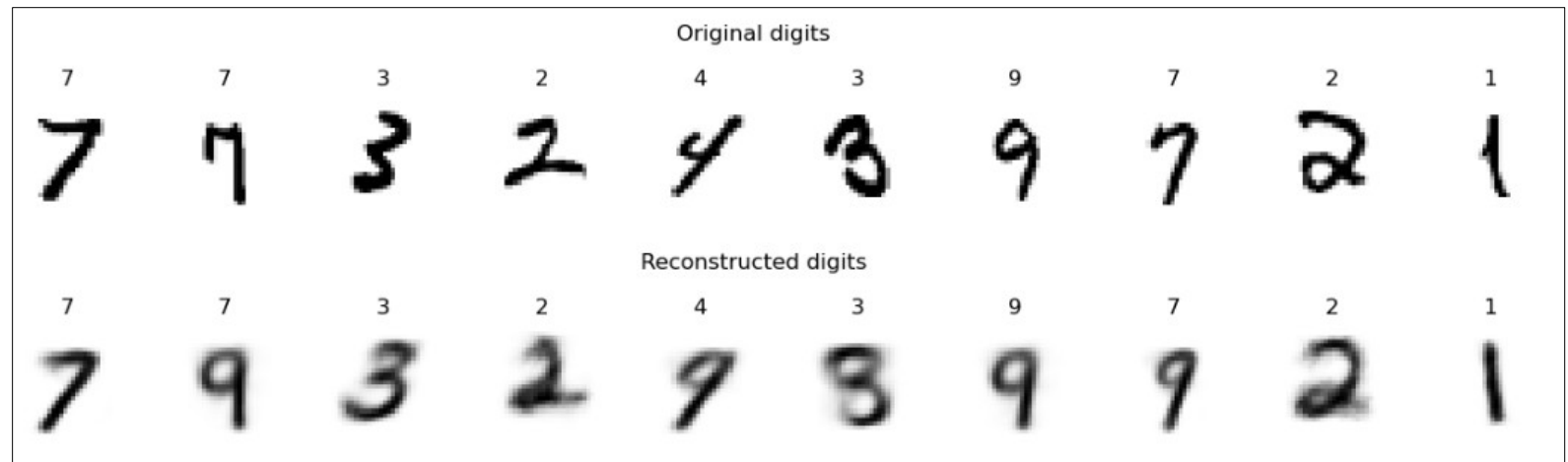
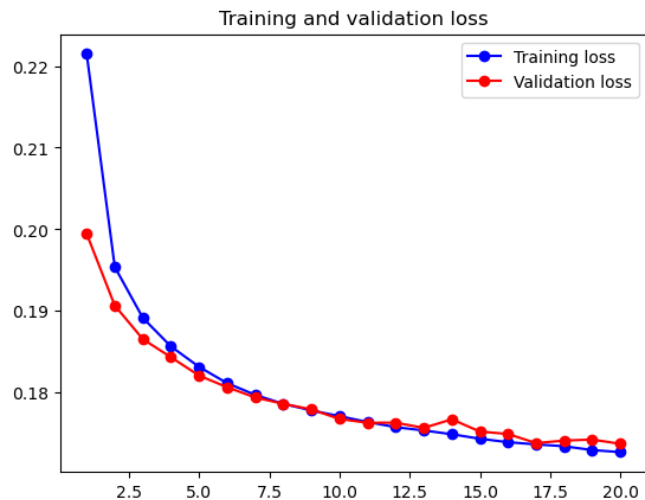
```
import keras
from tensorflow import expand_dims

autoencoder = keras.models.Sequential([encoder, decoder])

autoencoder.compile(optimizer='adam', loss = 'binary_crossentropy')

history = autoencoder.fit(x=x_train, y=x_train, batch_size=32, shuffle=True, epochs=20, validation_split=0.2)
# Use 20% of training samples for validation

autoencoder.save("U5A-models/UndercompleteAE.keras")
```



```

n_to_show = 5000
grid_size = 20
figsize = 8

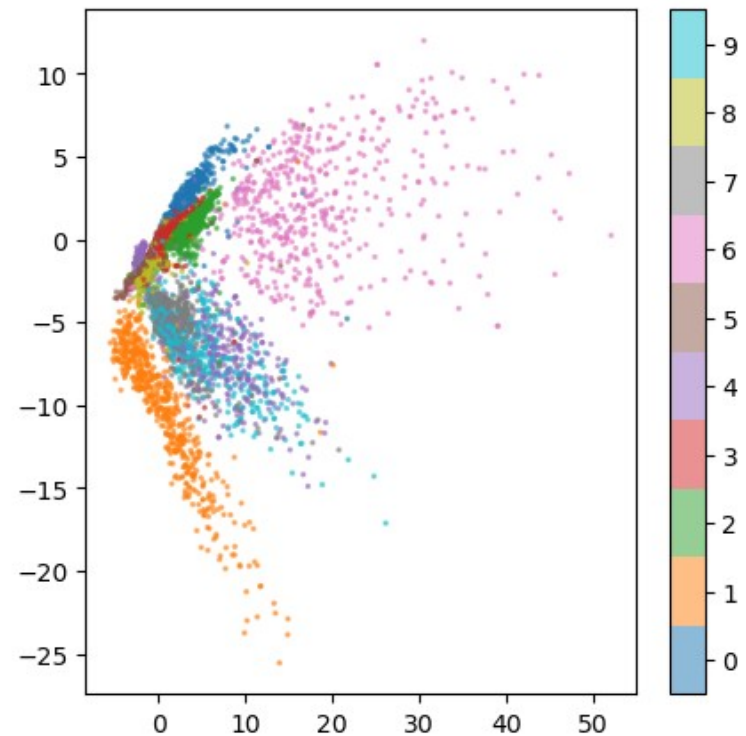
example_idx = np.random.choice(range(len(x_train)), n_to_show)
example_images = x_train[example_idx]
example_labels = y_train[example_idx]

z_points = encoder.predict(example_images)

plt.figure(figsize=(5, 5))
plt.scatter(z_points[:, 0], z_points[:, 1], cmap='tab10', c=example_labels, alpha=0.5, s=2)
plt.colorbar(values=range(10), ticks=range(10))

```

The latent space is 2D: we can represent the points after we use the encoder on the validation dataset

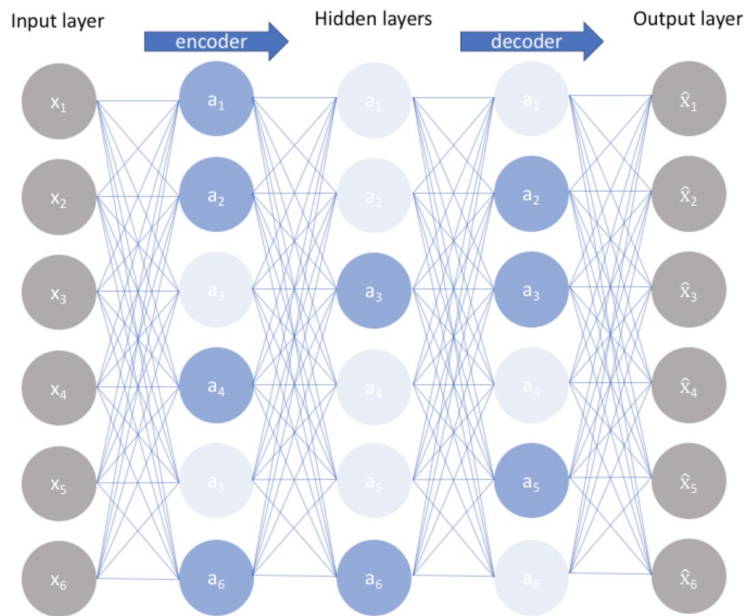


5B.3 Sparse autoencoders

Undercomplete autoencoders are regulated by the size of the bottleneck.

Sparse autoencoders are regulated **by reducing the number of active nodes** at each hidden layer.

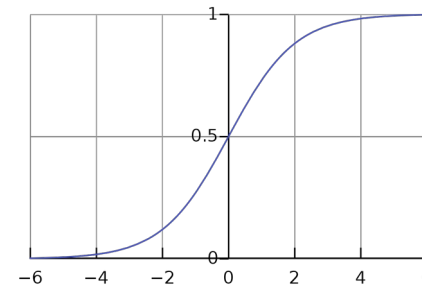
The activation of neurons in the hidden layers is penalized. This is equivalent to reduce the size with a bottleneck



Using sigmoids in hidden layers force the outputs to be zero or one, then reducing the number of active neurons

There are several strategies to do this:

- Using sigmoid function in hidden layers
- L1 regularization
- L2 regularization
- Kullback-Leibler (KL) divergence



L1 regularization: reconstruction loss is given by
$$L(x, \hat{x}) + \lambda \sum_i |a_i^{(h)}|$$

where $a_i^{(h)}$ are the activation coefficients at layer h and λ is a regularization parameter

This penalized layers with many active neurons

L2 regularization: similar to L1 loss is given by
$$L(x, \hat{x}) + \lambda \sum_i |a_i^{(h)}|^2$$

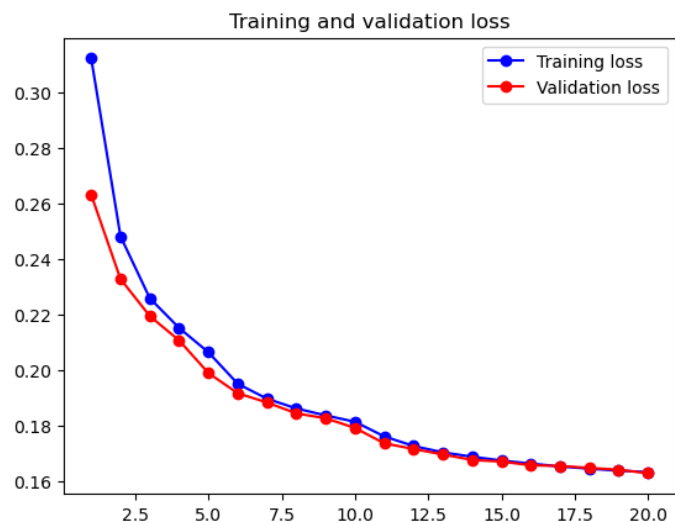
KL divergence: this method uses a sparsity parameter which defines the target for a layer so that the activations of its neurons follow a Bernoulli Distribution (a discrete distribution where a random variable takes value 1 with probability ρ and 0 with probability $(1-\rho)$)

The KL divergence method compares the ideal distribution with sparsity against the observed distribution $\hat{\rho}$ and sums this term to the loss function:

$$L(x, \hat{x}) + \lambda \sum_j KL(\rho || \hat{\rho})$$

These methods are applied to the hidden layers of a NN

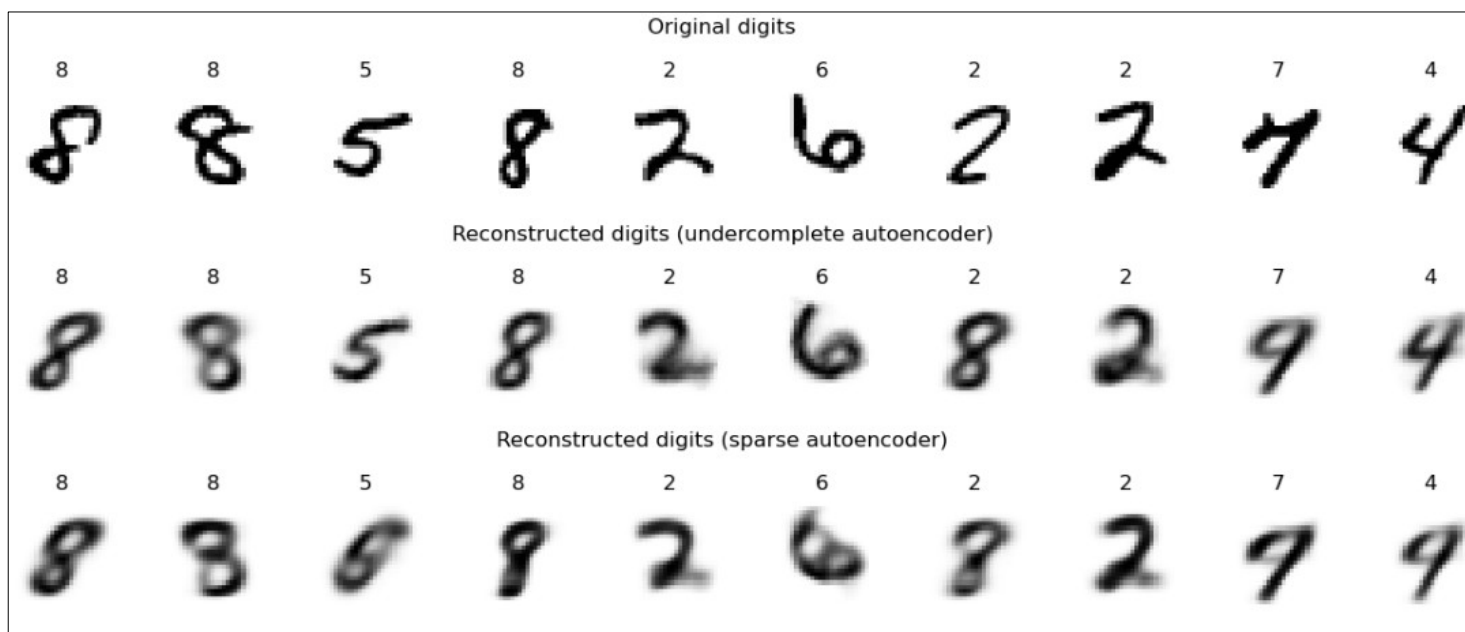
L1 and L2 regularization are implemented in Keras



```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.regularizers import l2

encoderSparse=Sequential()
encoderSparse.add(Dense(256, activation='relu', input_shape=(784,)))
encoderSparse.add(Dense(128, activity_regularizer=l2(0.001), activation='relu'))
encoderSparse.add(Dense(64, activity_regularizer=l2(0.001)))
encoderSparse.build()
```

```
decoderSparse=Sequential()
decoderSparse.add(Dense(128, activity_regularizer=l2(0.001), input_shape=(64,)))
decoderSparse.add(Dense(256, activity_regularizer=l2(0.001), activation='relu'))
decoderSparse.add(Dense(784, activation='sigmoid'))
decoderSparse.build()
```

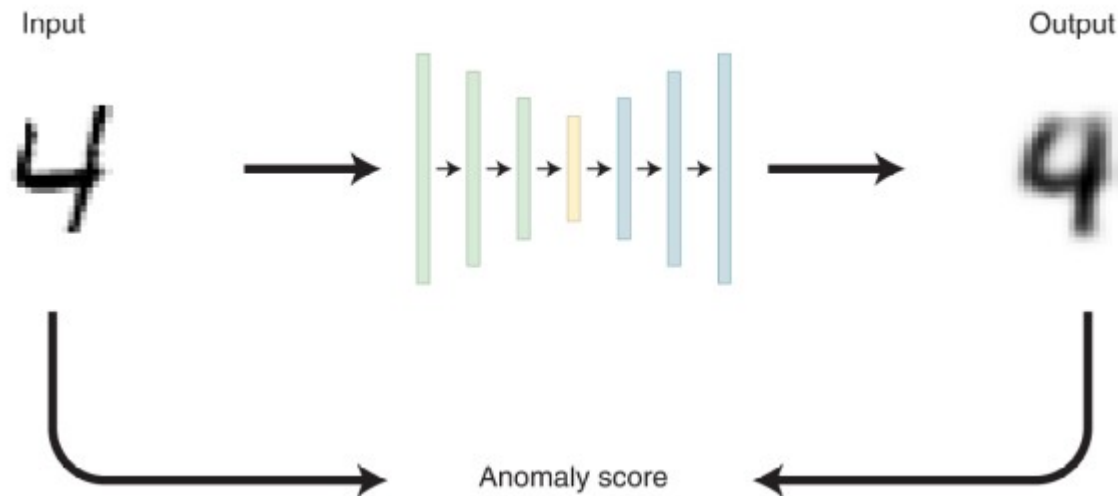


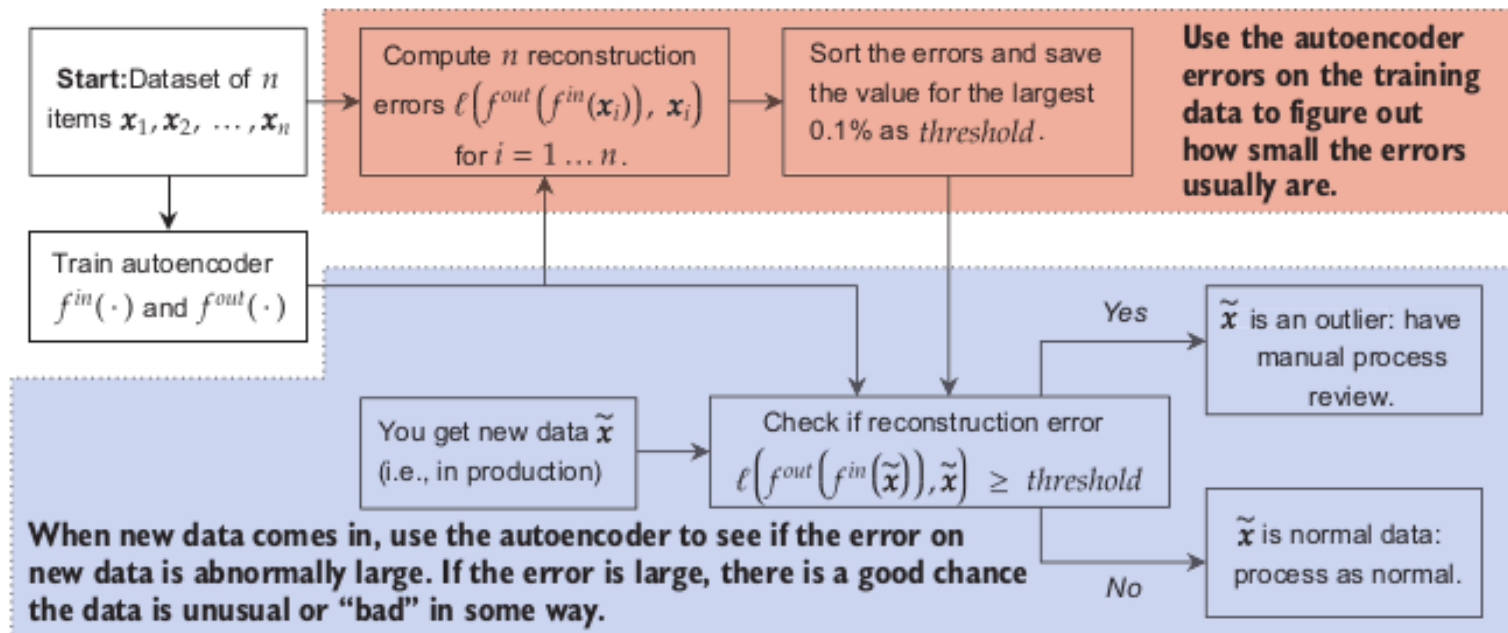
5B.4 Applying autoencoders for anomaly detection

Autoencoders try to minimize the reconstruction error as part of the training process.

One possibility is using the reconstruction error as a measure of the distance of a sample to the general characteristics of the training set.

In this case we are detecting outliers, that is, we are using the autoencoder as an **anomaly detector**.





Outlier detection can be done on training data or new testing data.

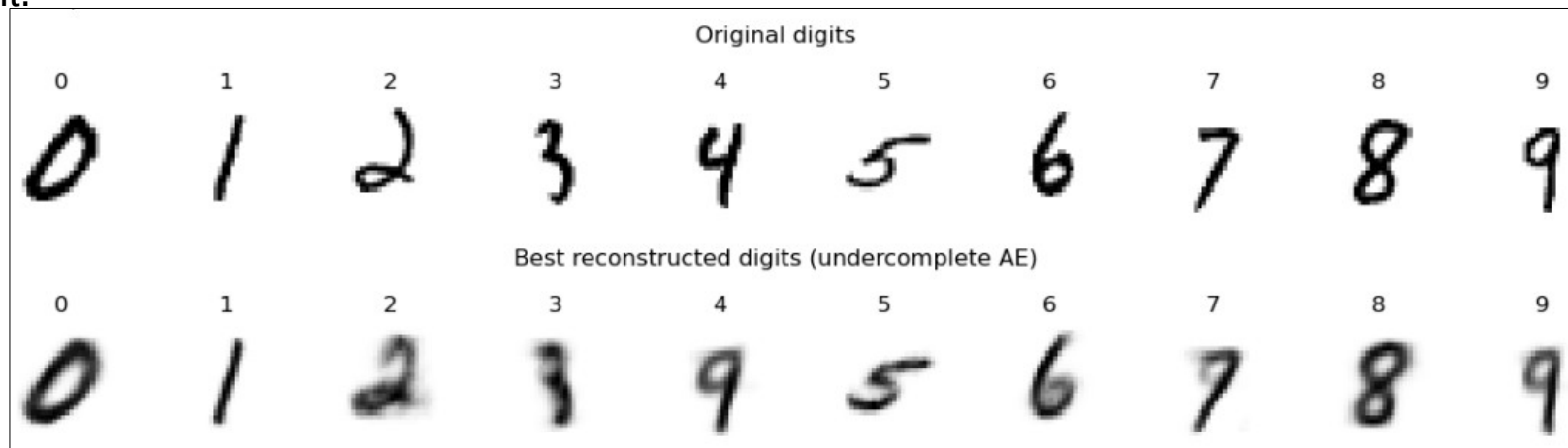
Example application of autoencoders to detect outliers:

- A dataset is used to train the autoencoder, and then you compute all the reconstruction errors.
- We estimate a rejection threshold using the validation set

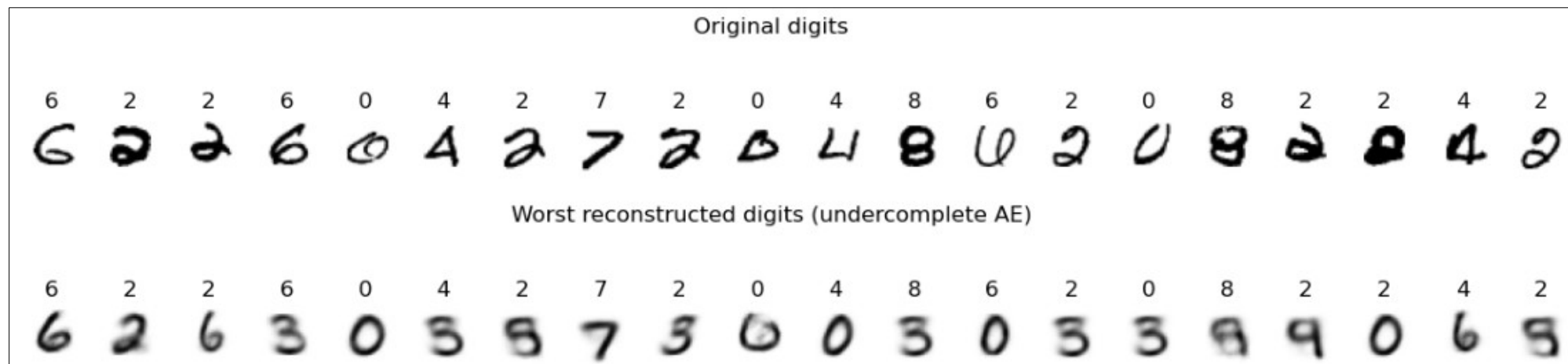
We remove outliers, or we flag them for later supervision

We study the digits that give the lowest and highest reconstruction error in the validation set using an undercomplete AE.

The lowest reconstruction errors are always given by 1's, so we take the best sample for each digit.

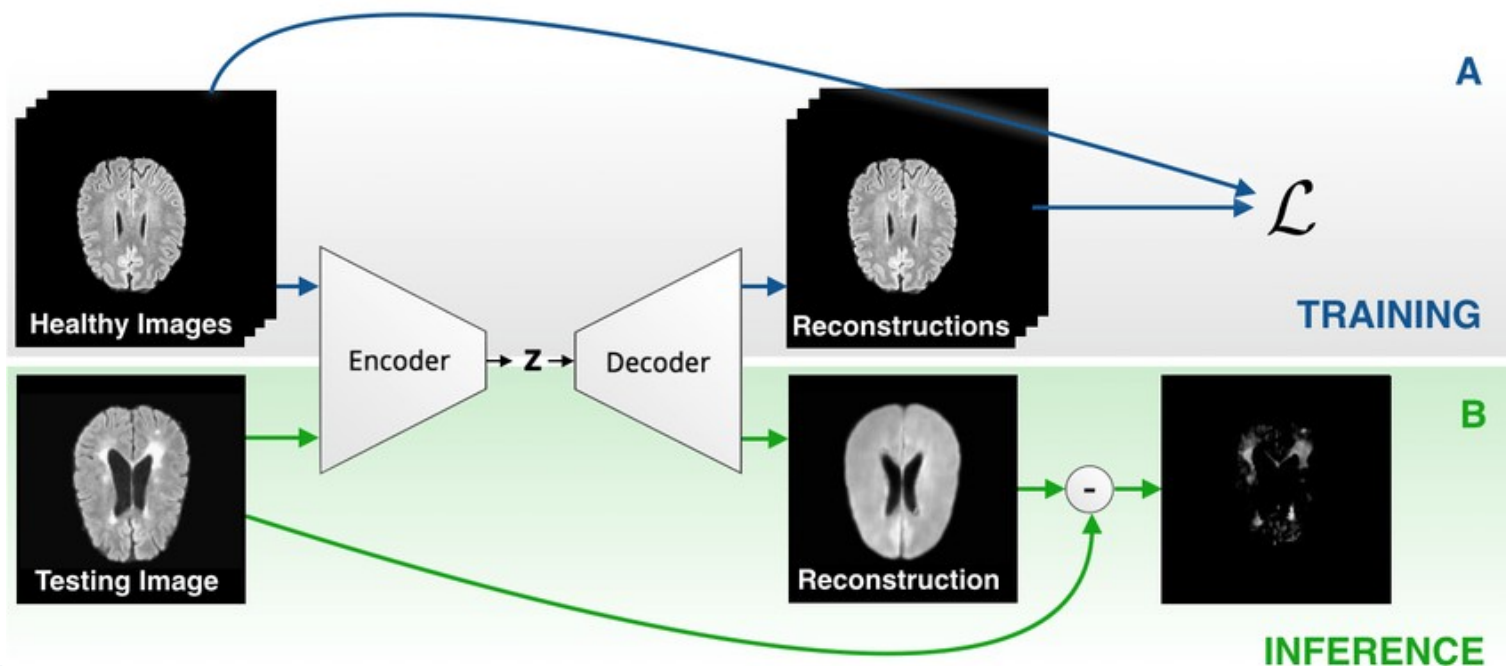
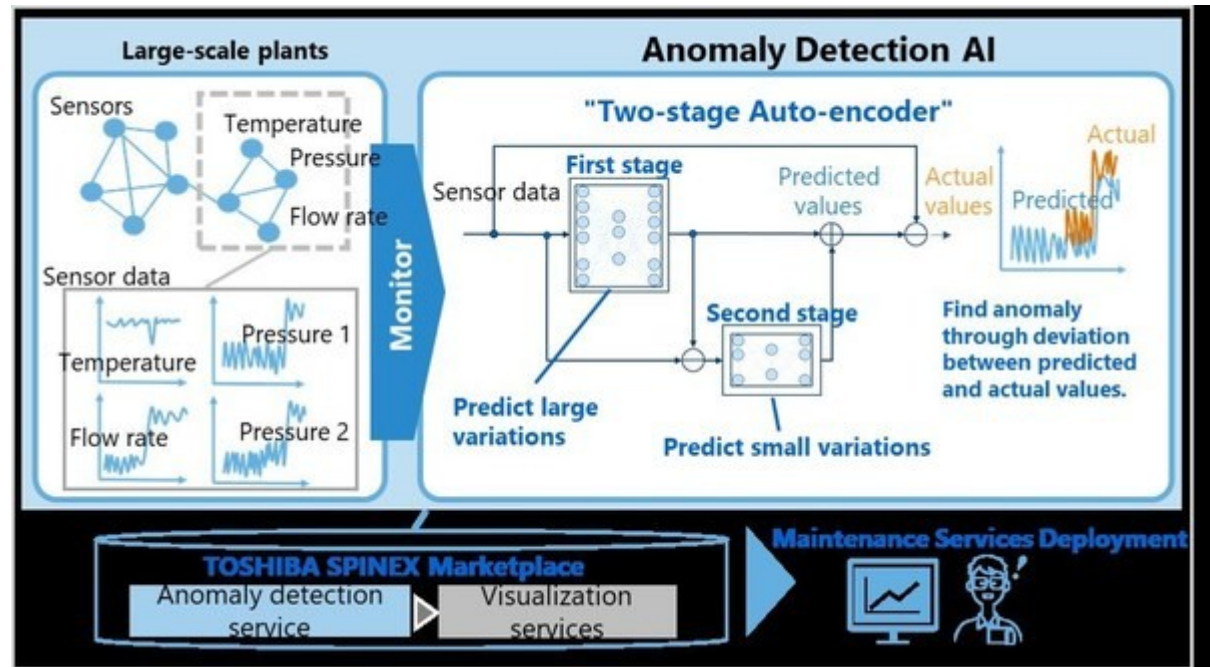


And these are the worst reconstructed digits:



Applications of anomaly detection:

- Network intrusion
- Medical systems
- Fraud detection
- Manufacturing
- Industrial processes



5B.5 Denoising autoencoders

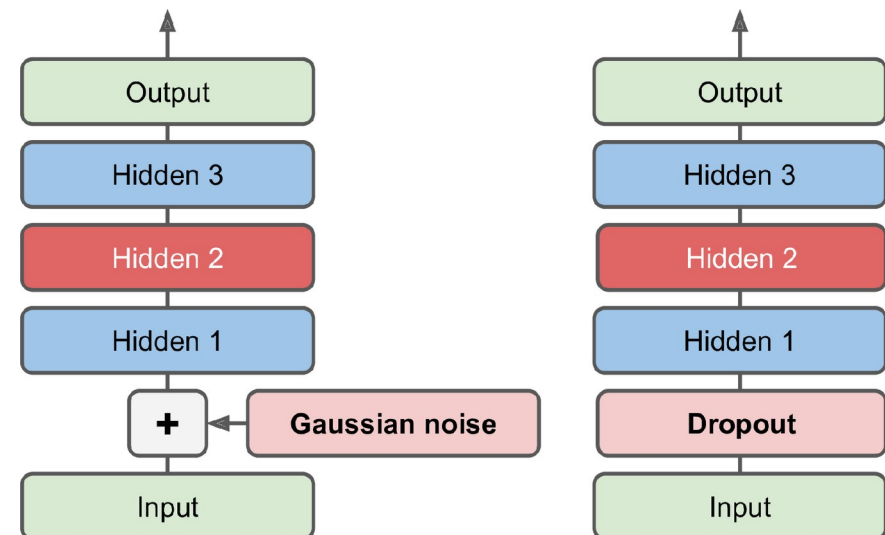
Autoencoders can be used to denoising or noise reduction, which is the process of removing noise from a signal.

Denoising autoencoders process a noisy set of samples, generating a clean set on the output side.

Important: they are trained by a corrupted version of the training set, but **the target at the output is the uncorrupted training data.**

Denoising autoencoders were not originally meant to reduce noise. It was a regularization technique:

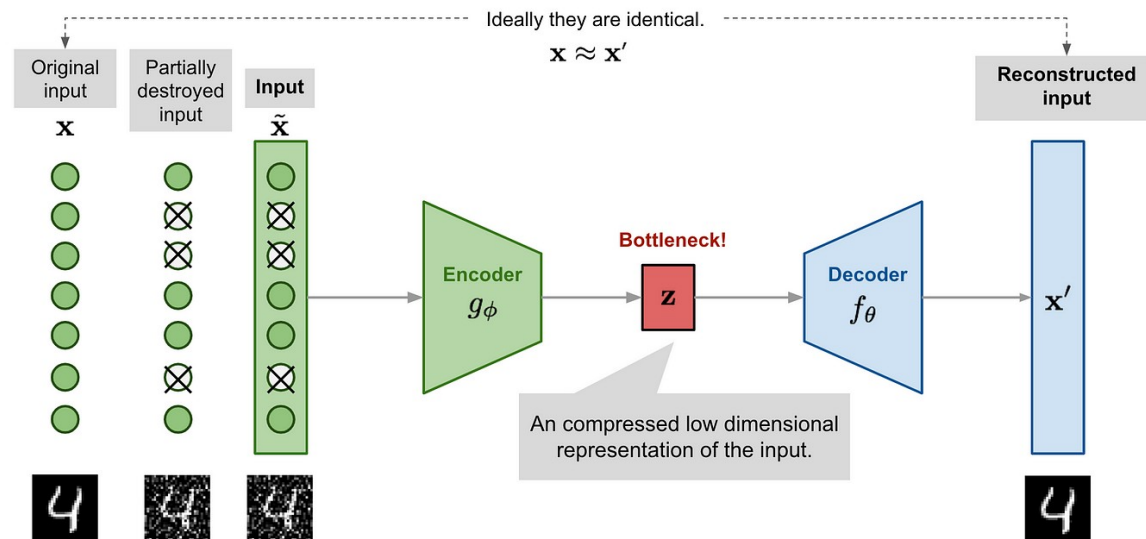
- The hidden layers to learn more robust filters
- Reduce the risk of overfitting
- Prevent the autoencoder to learn a simple identity function



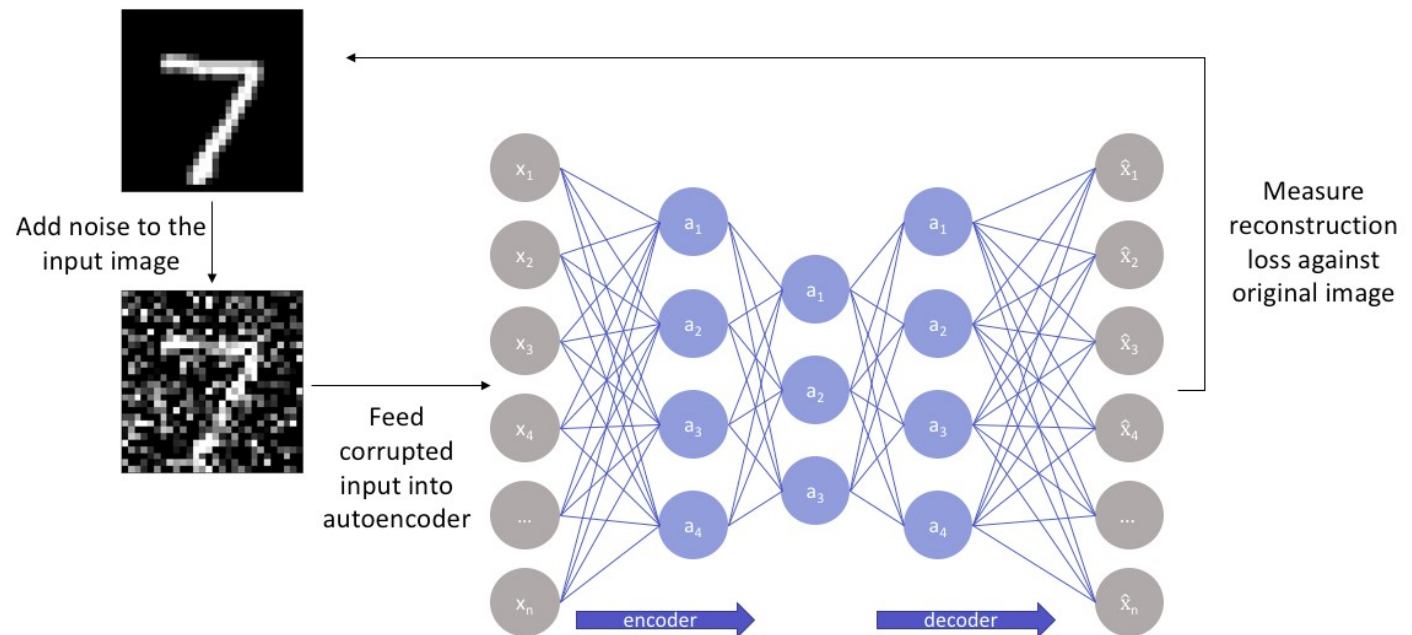
Two possibilities:

- Add noise
- Degrade the data

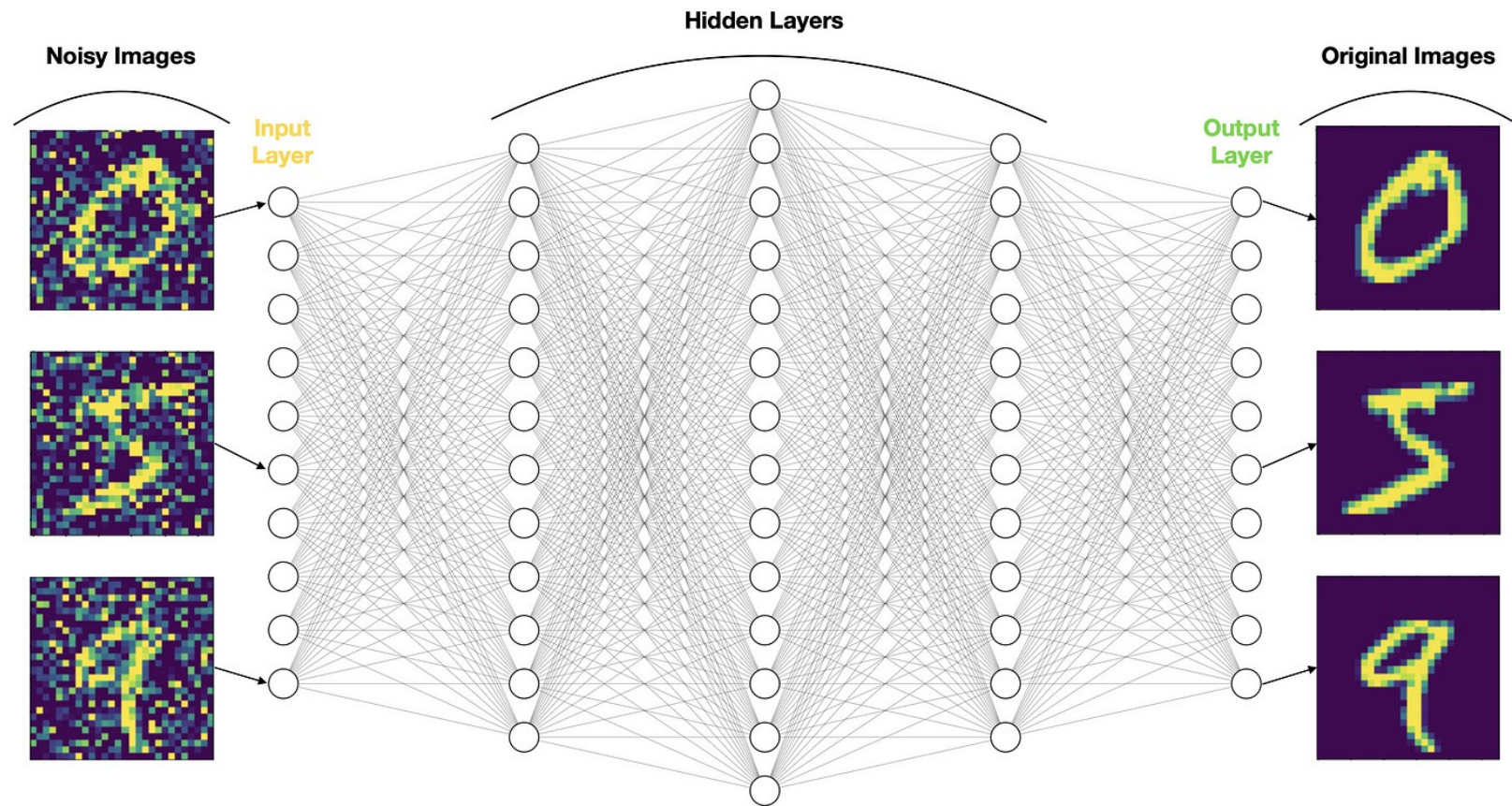
With degradation

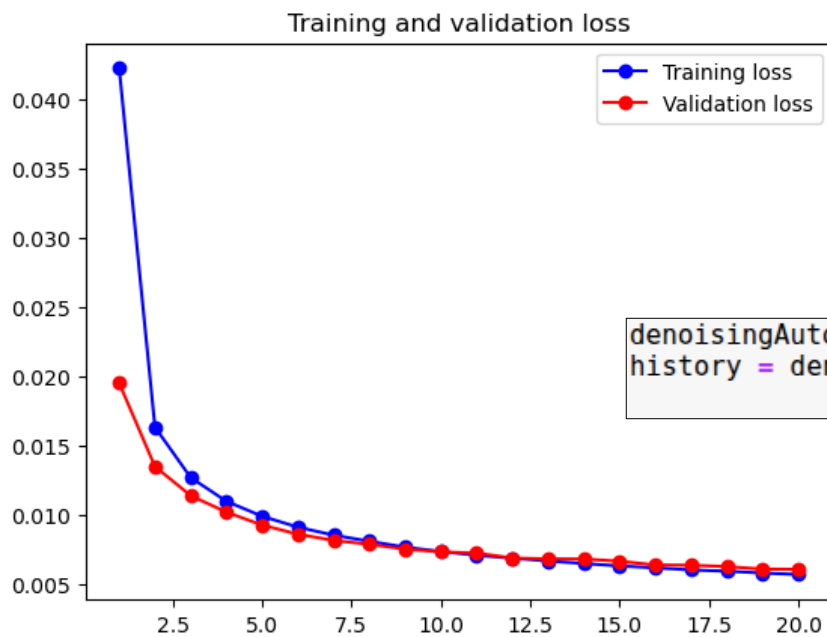


With added noise



As denoising autoencoders are able to convert noisy into clean data, they can be used to denoise data (hence their name)

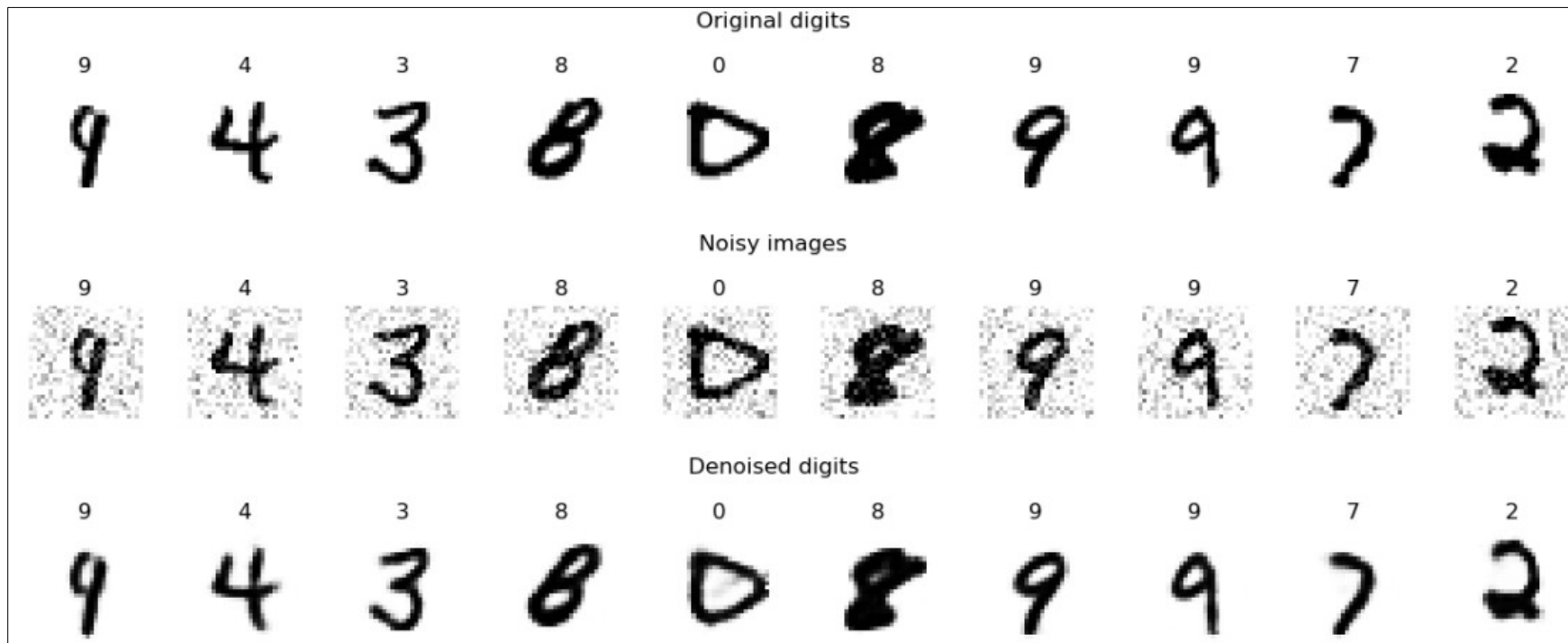




```
denoisingAutoencoder = Sequential()
denoisingAutoencoder.add(Dense(500, input_shape=(784,), activation='relu'))
denoisingAutoencoder.add(Dense(300, activation='relu'))
denoisingAutoencoder.add(Dense(100, activation='relu'))
denoisingAutoencoder.add(Dense(300, activation='relu'))
denoisingAutoencoder.add(Dense(500, activation='relu'))
denoisingAutoencoder.add(Dense(784, activation='sigmoid'))
denoisingAutoencoder.build()
```

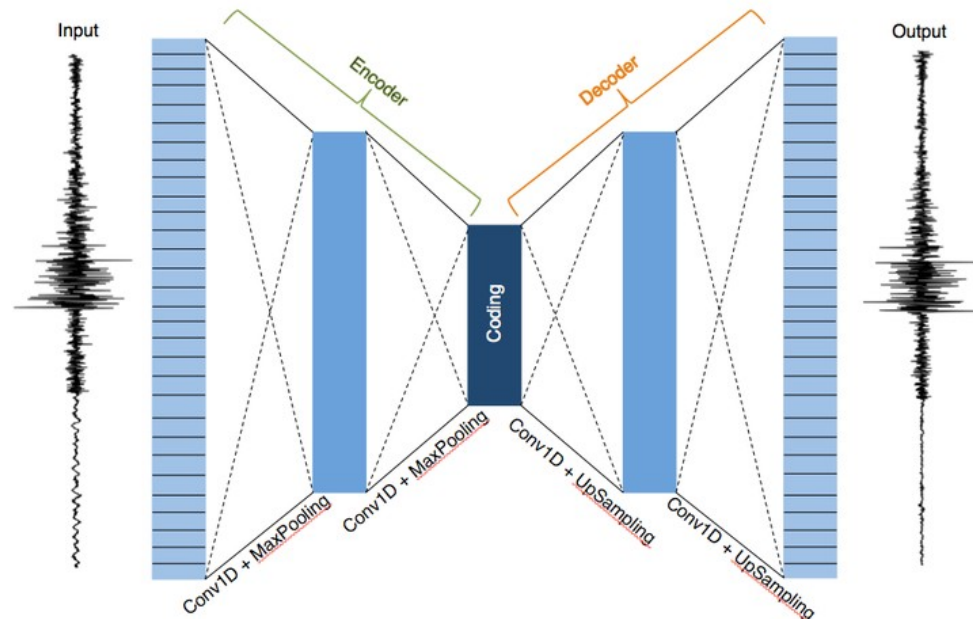
```
denoisingAutoencoder.compile(loss='mean_squared_error', optimizer='adam')
history = denoisingAutoencoder.fit(x_train_noisy, x_train,
                                   validation_data=(x_test_noisy, x_test), epochs=20, batch_size=200)
```

Training loss is calculated using noisy and clean samples



Removing Noise from Seismic Data with Denoising Autoencoders

Oct 7, 2019



<https://ngrayluna.github.io/project/denoiser/>

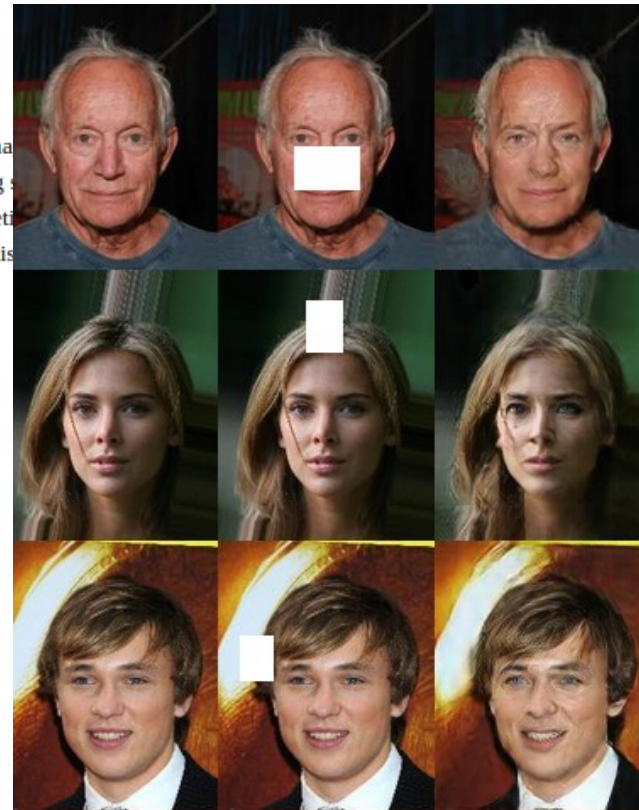
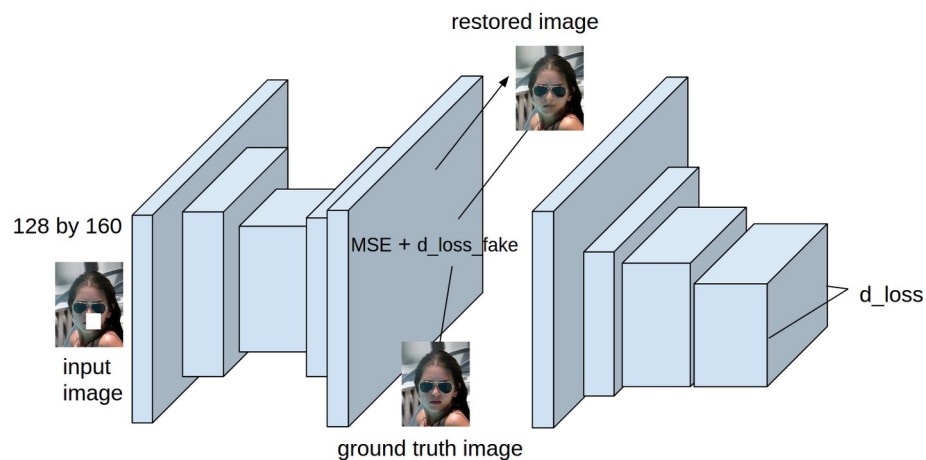
Applications of denoising autoencoders: inpainting images

Generative Adversarial Denoising Autoencoder for Face Completion

Avery Allen, Wenchen Li

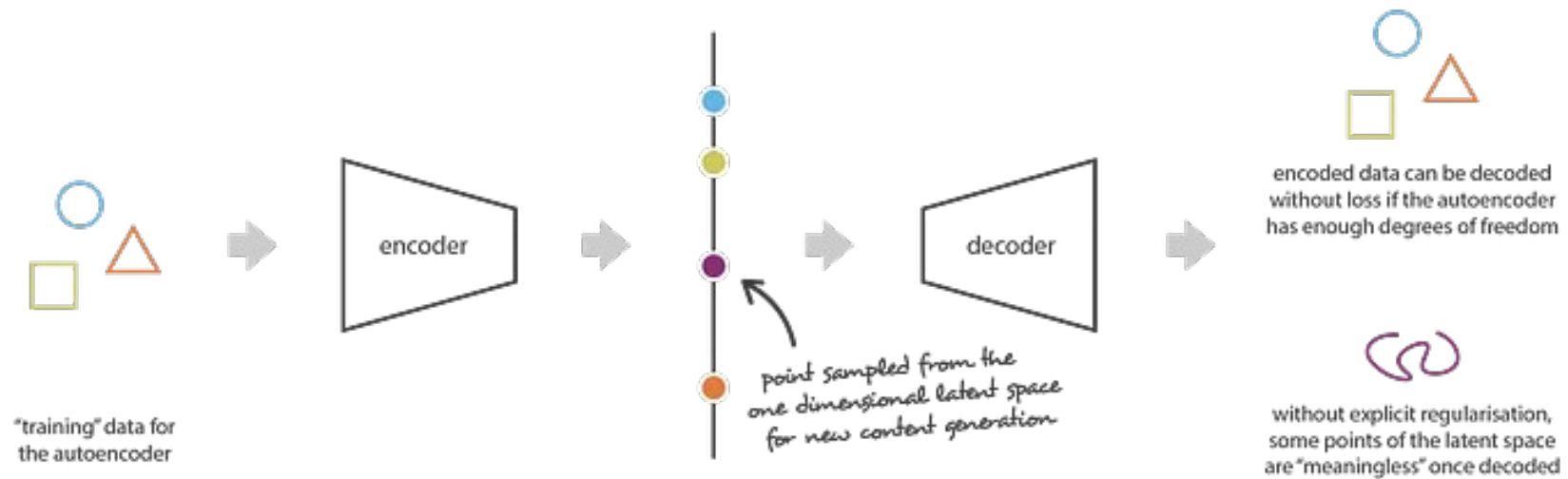
Project Overview

Our original project focus was creating a pipeline for photo restoration of portrait images. Many damaged solutions either provide unsatisfactory results, or require an advanced understanding of image editing scope to what we consider the most interesting subproblem of photo restoration: facial image completion. In this project, we focus on the following, given an image of a face with a rectangular section of the image set to be white, fill in the missing part.



https://faculty.cc.gatech.edu/~hays/7476/projects/Avery_Wenchen/

5B.6 Variational autoencoders (VAE)



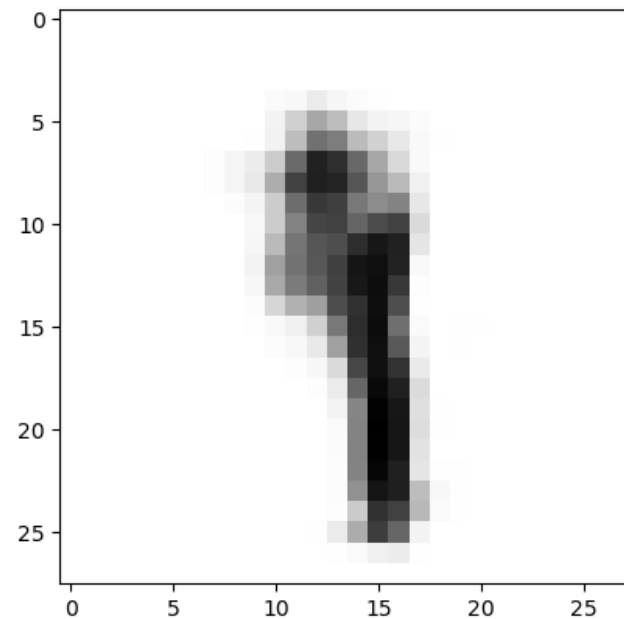
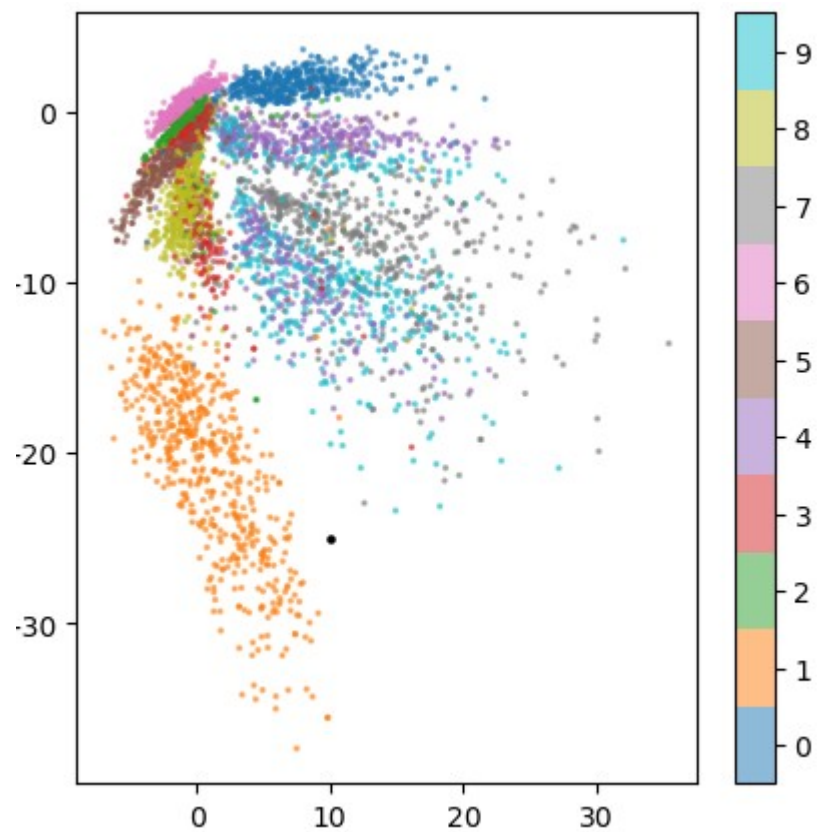
If we take a random point in the latent space, can we generate new data by decoding this point?. In a standard autoencoder, it is normally not possible

A VAE is an autoencoder where the latent space has some properties that allow us to generate data in a proper way.

Autoencoders' goal is to encode and decode with the minimum losses, and not to organize the latent space in any way

If we want to use AEs to generate data, we have to assure that the latent space is well organized

In an standard autoencoder, the latent space is disorganized

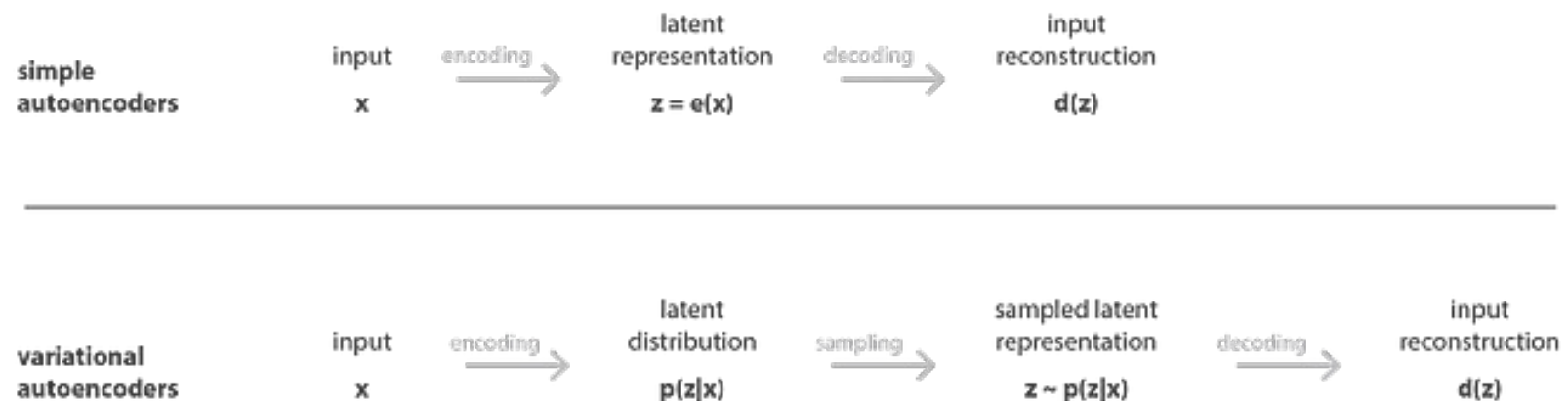


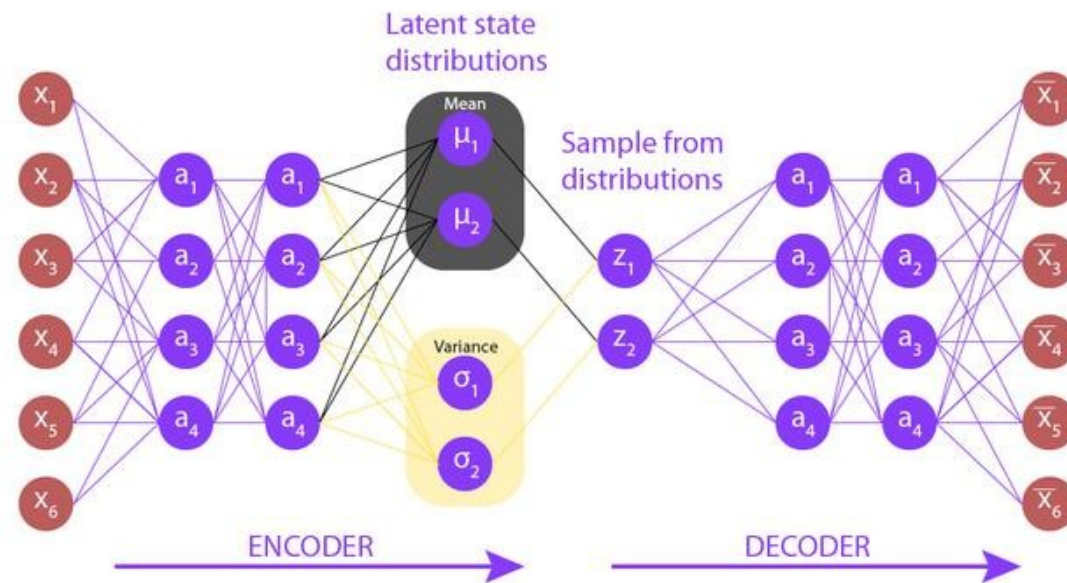
In order to use the autoencoder for generation, the latent space should be regular enough. This is achieved with regularization

In VAEs, instead of encoding the input as single points, **they are encoded as a distribution** over the latent space.

The training process is:

- 1) The input is encoded as a distribution over the latent space
- 2) A point is sampled from this distribution
- 3) The point is decoded and the reconstruction error is computed
- 4) The reconstruction error is backpropagated through the NN





The output of the encoder is a multivariate gaussian that represents a compressed version of the input.

The dimension of the multivariate gaussian is the dimension of the latent space.

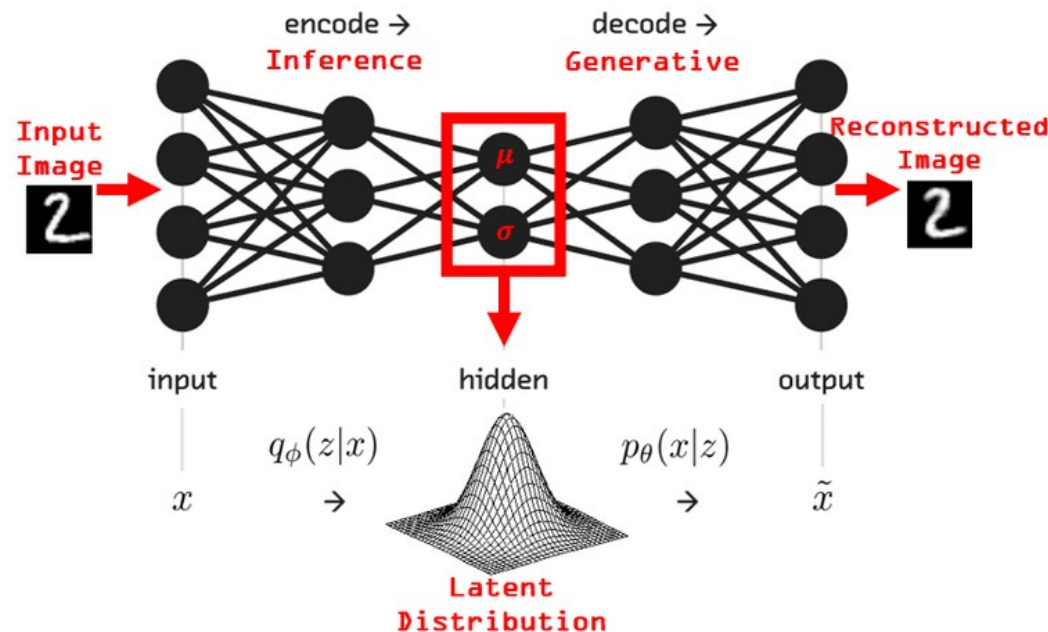
How can be ensured that the encoder maps to a standard Normal distribution $N(0,1)$?

Introduce a regularization term based on the KL (Kullback-Leiber) divergence, that measures the distances between probability distributions.

$$Loss = L(x, \hat{x}) + D_{KL} \quad D_{KL} [N(\mu, \sigma) || N(0, 1)] = \frac{1}{2} \sum_{i=1}^n (1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2)$$

The reconstruction error is going to have two components:

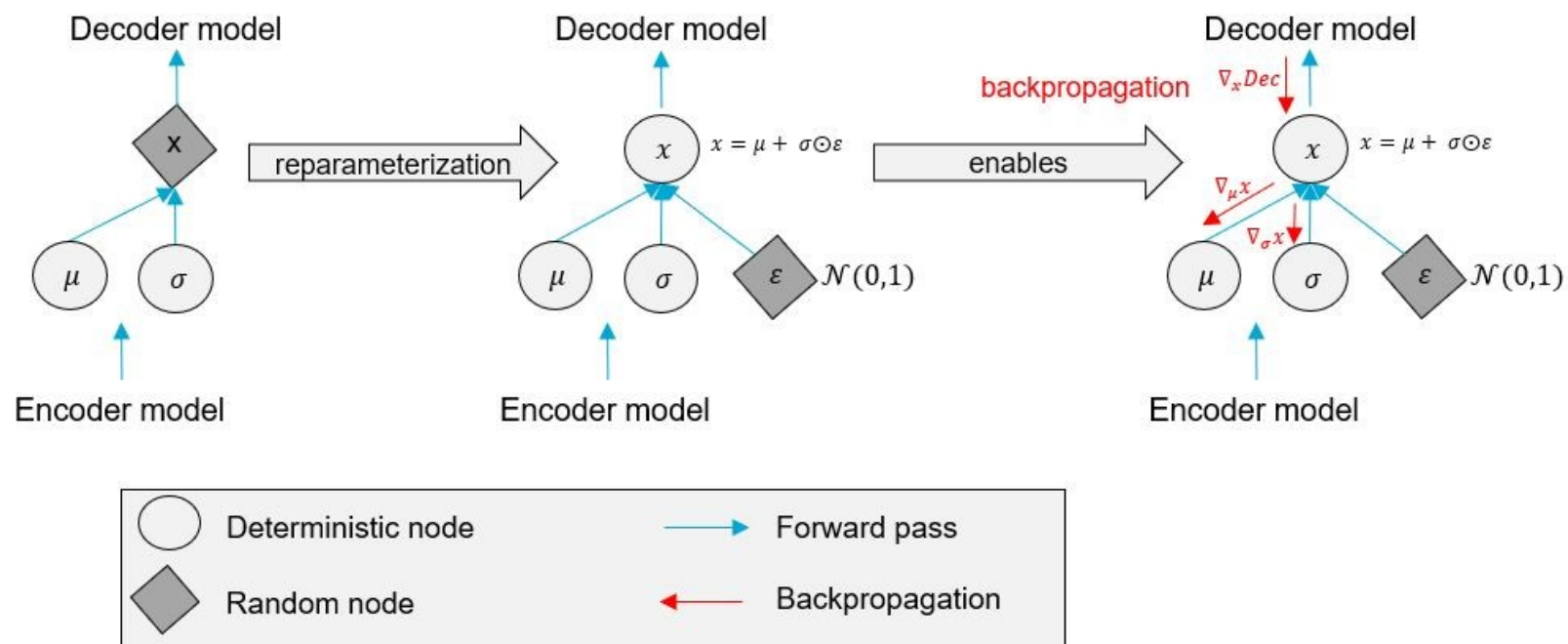
- A standard reconstruction error
- A regularization component term defined as the Kulback-Leibler divergence between the distribution and the standard gaussian distribution (0,1)



Can we backpropagate when there is a sampling in an intermediate layer?

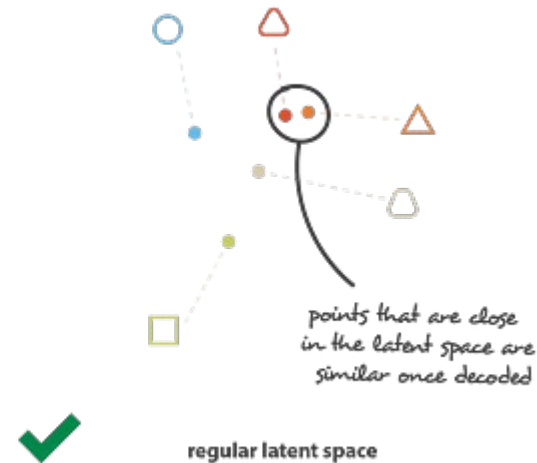
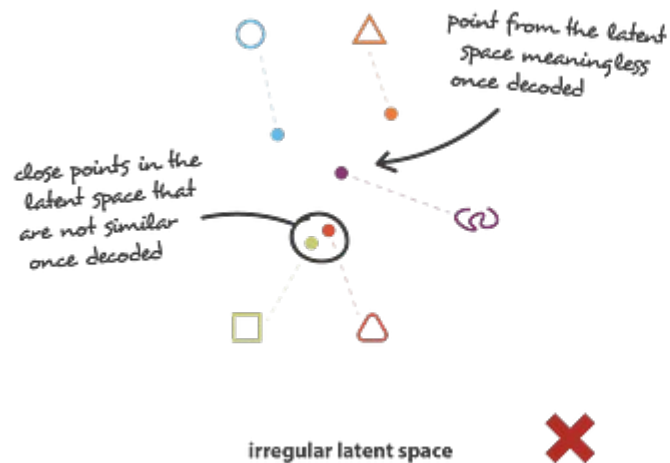
The answer is no, we can't

This can be solved using the **reparameterization trick**: we sample a hidden state x with $x = \mu + \sigma \epsilon$, where ϵ is a standard gaussian $\mathcal{N}(0,1)$

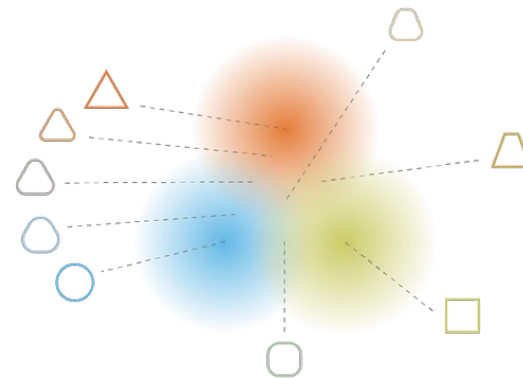


This regularization has two effects on the latent space:

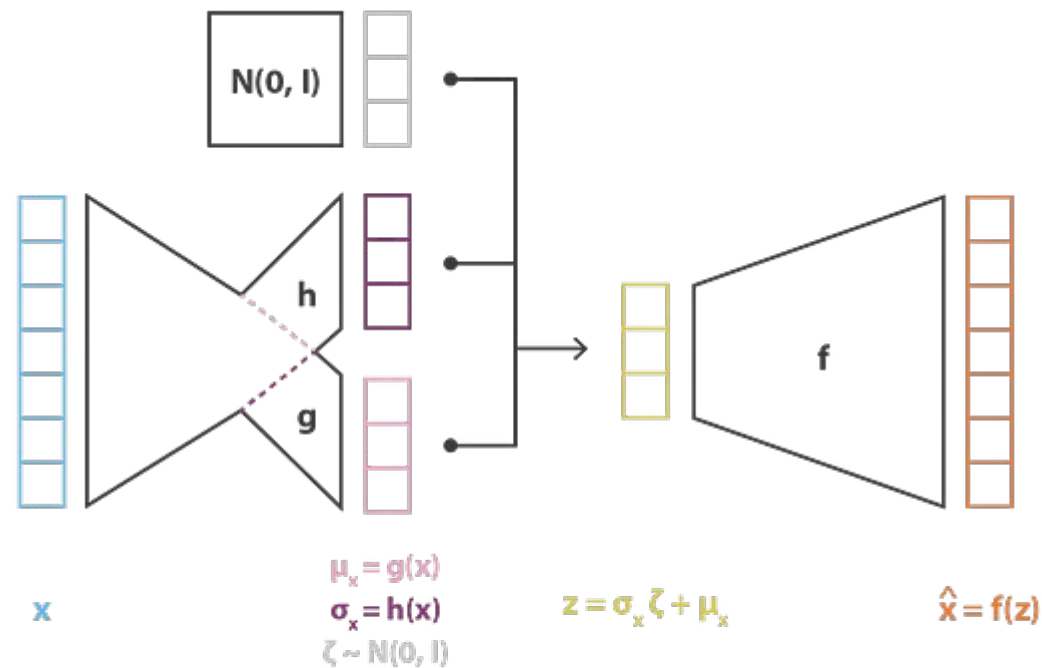
- **Continuity**: two close points in the latent space should give similar contents once decoded
- **Completeness**: a random point in the latent space should give meaningful content once decoded



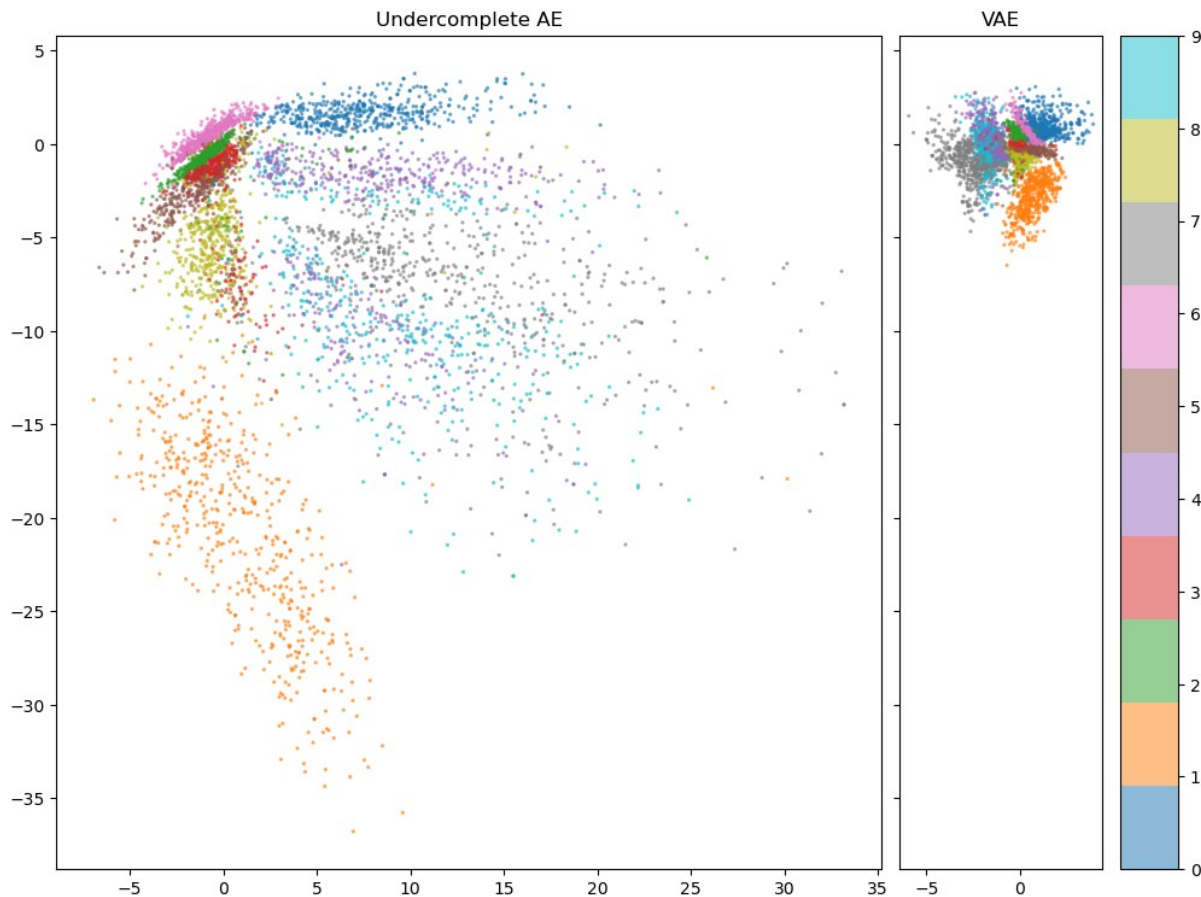
This tends to create a **gradient** over the information encoded in the latent space



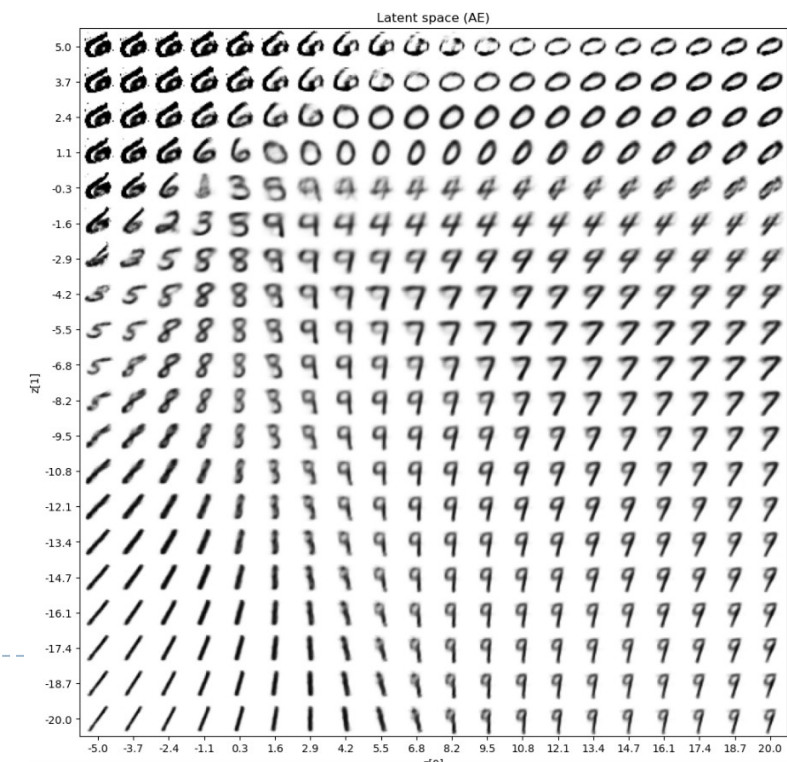
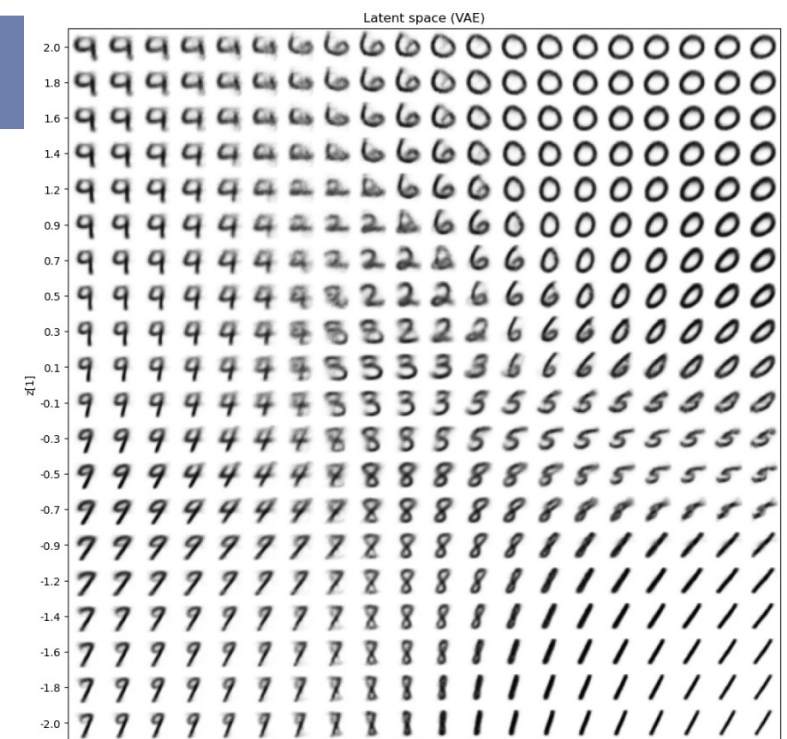
This is all summarized on the following diagram:



$$\text{loss} = C \|x - \hat{x}\|^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)] = C \|x - f(z)\|^2 + \text{KL}[N(g(x), h(x)), N(0, I)]$$



Let's see some code....



```
latent_dim = 2

encoder_inputs = Input(shape=(28, 28, 1))
x = Conv2D(32, 3, activation="relu", strides=2, padding="same")(encoder_inputs)
x = Conv2D(64, 3, activation="relu", strides=2, padding="same")(x)
x = Flatten()(x)
x = Dense(16, activation="relu")(x)
z_mean = Dense(latent_dim, name="z_mean")(x)
z_log_var = Dense(latent_dim, name="z_log_var")(x)
z = Sampling()([z_mean, z_log_var])
encoderVAE = Model(encoder_inputs, [z_mean, z_log_var, z], name="encoderVAE")
```

Samples from a 2D gaussian

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 28, 28, 1)	0	-
conv2d (Conv2D)	(None, 14, 14, 32)	320	input_layer[0][0]
conv2d_1 (Conv2D)	(None, 7, 7, 64)	18,496	conv2d[0][0]
flatten (Flatten)	(None, 3136)	0	conv2d_1[0][0]
dense (Dense)	(None, 16)	50,192	flatten[0][0]
z_mean (Dense)	(None, 2)	34	dense[0][0]
z_log_var (Dense)	(None, 2)	34	dense[0][0]
sampling (Sampling)	(None, 2)	0	z_mean[0][0], z_log_var[0][0]

Total params: 69,076 (269.83 KB)

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 2)	0
dense_1 (Dense)	(None, 3136)	9,408
reshape (Reshape)	(None, 7, 7, 64)	0
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 64)	36,928
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 32)	18,464
conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 1)	289

Total params: 65,089 (254.25 KB)

Strides and padding to increase size

```
latent_inputs = keras.Input(shape=(latent_dim, ))
x = Dense(7 * 7 * 64, activation="relu")(latent_inputs)
x = Reshape((7, 7, 64))(x)
x = Conv2DTranspose(64, 3, activation="relu", strides=2, padding="same")(x)
x = Conv2DTranspose(32, 3, activation="relu", strides=2, padding="same")(x)
decoder_outputs = Conv2DTranspose(1, 3, activation="sigmoid", padding="same")(x)
decoderVAE = Model(latent_inputs, decoder_outputs, name="decoderVAE")
```

```

class VAE(keras.Model):
    def __init__(self, encoderVAE, decoderVAE, **kwargs):
        super(VAE, self).__init__(**kwargs)
        self.encoderVAE = encoderVAE
        self.decoderVAE = decoderVAE

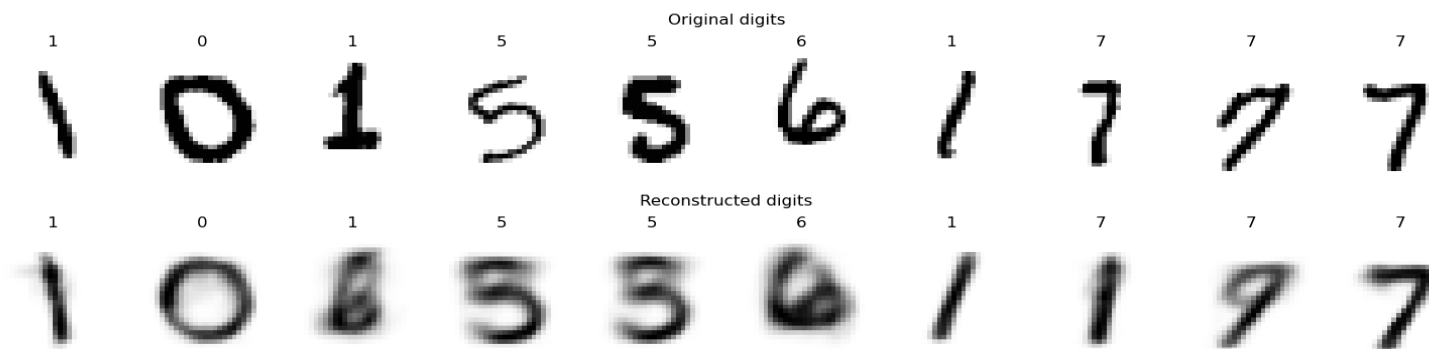
    def train_step(self, data):
        if isinstance(data, tuple):
            data = data[0]
        with tf.GradientTape() as tape:
            z_mean, z_log_var, z = encoderVAE(data)
            reconstruction = decoderVAE(z)
            reconstruction_loss = tf.reduce_mean(
                keras.losses.binary_crossentropy(data, reconstruction)
            )
            reconstruction_loss *= 28 * 28
            kl_loss = 1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var)
            kl_loss = tf.reduce_mean(kl_loss)
            kl_loss *= -0.5
            total_loss = reconstruction_loss + kl_loss
        grads = tape.gradient(total_loss, self.trainable_weights)
        self.optimizer.apply_gradients(zip(grads, self.trainable_weights))
        return {
            "loss": total_loss,
            "reconstruction_loss": reconstruction_loss,
            "kl_loss": kl_loss,
        }

```

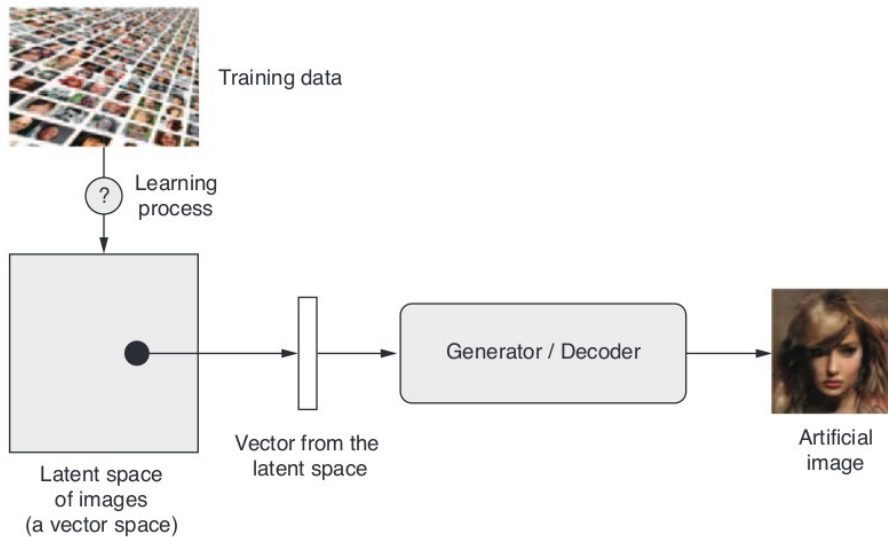
The log-variance is used instead of variance (to avoid problems with negative numbers)

$$D_{KL} [N(\mu, \sigma) || N(0, 1)] = \frac{1}{2} \sum_{i=1}^n (1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2)$$

$$Loss = L(x, \hat{x}) + D_{KL}$$



5A.8 Using AEs to generate new faces



Both AEs and VAEs allow us to develop a low-dimensional latent space of representations where any point can be mapped to a realistic looking sample

VAEs create well structured latent spaces, where specific directions encode a meaningful axis of variation in the data

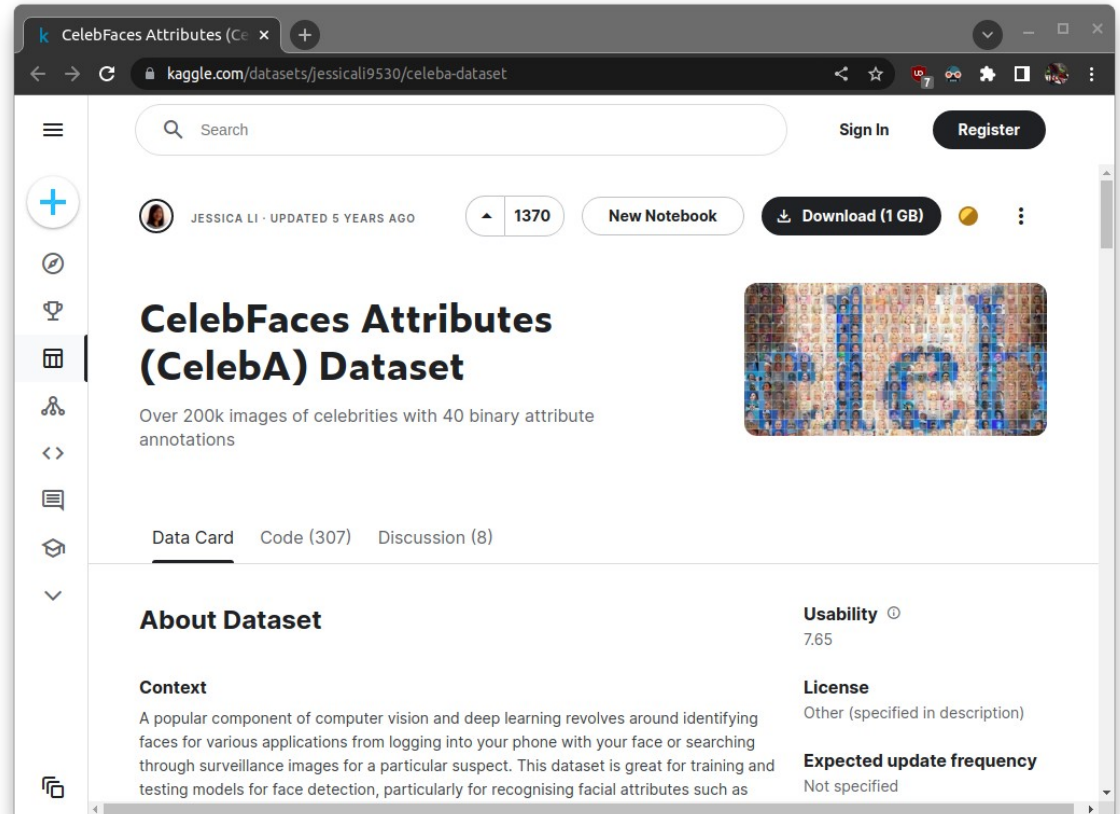
First, we are going to use standard AEs, and then compare them against VAEs to generate new faces.

The dataset

Content

Overall

- 202,599 number of face images of various celebrities
- 10,177 unique identities, but names of identities are not given
- 40 binary attribute annotations per image
- 5 landmark locations



The dataset is very large, so we are going to create and **ImageDataGenerator** object (that is an iterator) and use its **flow_from_directory** function instead of loading all the dataset.

Encoder: a stack of convolutional layers and bottleneck's dimension is 200

Decoder: basically a mirror image of the encoder

Loss function is RMSE, and the **Adam optimizer** is used

Let's see some code....

```

import tensorflow.keras.backend as K
# ENCODER
def build_encoder(input_dim, output_dim, conv_filters, conv_kernel_size, conv_strides):
    # Number of Conv layers
    n_layers = len(conv_filters)

    # Define model input
    encoder_input = Input(shape = input_dim, name = 'encoder_input')
    x = encoder_input

    # Add convolutional layers
    for i in range(n_layers):
        x = Conv2D(filters = conv_filters[i],
                    kernel_size = conv_kernel_size[i],
                    strides = conv_strides[i],
                    padding = 'same',
                    name = 'encoder_conv_' + str(i)
                    )(x)

        x = LeakyReLU()(x)

    # Required for reshaping latent vector while building Decoder
    shape_before_flattening = K.int_shape(x)[1:]

    x = Flatten()(x)

    # Define model output
    encoder_output = Dense(output_dim, name = 'encoder_output')(x)

    return encoder_input, encoder_output,
        shape_before_flattening,
        Model(encoder_input, encoder_output)

encoder_input, encoder_output, shape_before_flattening, encoder = build_encoder(
    input_dim = INPUT_DIM,
    output_dim = Z_DIM,
    conv_filters = [32, 64, 64, 64],
    conv_kernel_size = [3,3,3,3],
    conv_strides = [2,2,2,2])

```

Layer (type)	Output Shape	Param #
encoder_input (InputLayer)	(None, 128, 128, 3)	0
encoder_conv_0 (Conv2D)	(None, 64, 64, 32)	896
leaky_re_lu (LeakyReLU)	(None, 64, 64, 32)	0
encoder_conv_1 (Conv2D)	(None, 32, 32, 64)	18,496
leaky_re_lu_1 (LeakyReLU)	(None, 32, 32, 64)	0
encoder_conv_2 (Conv2D)	(None, 16, 16, 64)	36,928
leaky_re_lu_2 (LeakyReLU)	(None, 16, 16, 64)	0
encoder_conv_3 (Conv2D)	(None, 8, 8, 64)	36,928
leaky_re_lu_3 (LeakyReLU)	(None, 8, 8, 64)	0
flatten (Flatten)	(None, 4096)	0
encoder_output (Dense)	(None, 200)	819,400

Total params: 912,648 (3.48 MB)

```
def build_decoder(input_dim, shape_before_flattening, conv_filters, conv_kernel_size, conv_strides):
```

```
    # Number of Conv layers
```

```
    n_layers = len(conv_filters)
```

```
    # Define model input
```

```
    decoder_input = Input(shape = (input_dim,) , name = 'decoder_input')
```

```
    # To get an exact mirror image of the encoder
```

```
    x = Dense(int(np.prod(shape_before_flattening)))(decoder_input)
```

```
    x = Reshape(shape_before_flattening)(x)
```

```
    # Add convolutional layers
```

```
    for i in range(n_layers):
```

```
        x = Conv2DTranspose(
            filters = conv_filters[i],
            kernel_size = conv_kernel_size[i],
            strides = conv_strides[i],
            padding = 'same',
            name = 'decoder_conv_' + str(i)
        )(x)
```

```
    # Adding a sigmoid layer at the end to restrict the outputs
```

```
    # between 0 and 1
```

```
    if i < n_layers - 1:
```

```
        x = LeakyReLU()(x)
```

```
    else:
```

```
        x = Activation('sigmoid')(x)
```

```
    # Define model output
```

```
    decoder_output = x
```

```
    return decoder_input, decoder_output, Model(decoder_input, decoder_output)
```

```
decoder_input, decoder_output, decoder = build_decoder(input_dim = Z_DIM,
    shape_before_flattening = shape_before_flattening,
    conv_filters = [64,64,32,3],
    conv_kernel_size = [3,3,3,3],
    conv_strides = [2,2,2,2]
)
```

Layer (type)	Output Shape	Param #
decoder_input (InputLayer)	(None, 200)	0
dense (Dense)	(None, 4096)	823,296
reshape (Reshape)	(None, 8, 8, 64)	0
decoder_conv_0 (Conv2DTranspose)	(None, 16, 16, 64)	36,928
leaky_re_lu_4 (LeakyReLU)	(None, 16, 16, 64)	0
decoder_conv_1 (Conv2DTranspose)	(None, 32, 32, 64)	36,928
leaky_re_lu_5 (LeakyReLU)	(None, 32, 32, 64)	0
decoder_conv_2 (Conv2DTranspose)	(None, 64, 64, 32)	18,464
leaky_re_lu_6 (LeakyReLU)	(None, 64, 64, 32)	0
decoder_conv_3 (Conv2DTranspose)	(None, 128, 128, 3)	867
activation (Activation)	(None, 128, 128, 3)	0

Total params: 916,483 (3.50 MB)



```

simple_autoencoder_input = encoder_input

simple_autoencoder_output = decoder(encoder_output)

# Input to the combined model will be the input to the encoder.
# Output of the combined model will be the output of the decoder.
simple_autoencoder = Model(simple_autoencoder_input, simple_autoencoder_output)

```

```

LEARNING_RATE = 0.0005
N_EPOCHS = 25

adam_optimizer = Adam(learning_rate = LEARNING_RATE)

def r_loss(y_true, y_pred):
    return K.mean(K.square(y_true - y_pred), axis = [1,2,3])

simple_autoencoder.compile(optimizer=adam_optimizer, loss = r_loss)

simple_autoencoder.fit(dataset, epochs = N_EPOCHS)

simple_autoencoder.save(os.path.join(WEIGHTS_FOLDER, 'AE_model.keras'))

```

```
def plot_compare(images, add_noise=False):
```

```
    n_to_show = images.shape[0]
```

```
    if add_noise:
        encodings = encoder.predict(images)
        encodings += np.random.normal(0.0, 1.0, size = (n_to_show,200))
        reconst_images = decoder.predict(encodings)
```

```
    else:
        reconst_images = simple_autoencoder.predict(images)
```

```
    fig = plt.figure(figsize=(12, 3))
    fig.subplots_adjust(left=0.05, bottom=0.05,
                        right=0.95, top=0.95,
                        hspace=0.1, wspace=0.1)
```

```
    for i in range(n_to_show):
        #img = images[i].squeeze()
        img = images[i]
        sub = fig.add_subplot(2, n_to_show, i+1)
        sub.axis('off')
        sub.imshow(img)
```

```
    for i in range(n_to_show):
        img = reconst_images[i].squeeze()
        sub = fig.add_subplot(2, n_to_show, i+n_to_show+1)
        sub.axis('off')
        sub.imshow(img)
```

```
    plt.show()
```

```
example_images = example_batch[:8]
```

```
plot_compare(example_images)
```

If True, adds noise in latent space

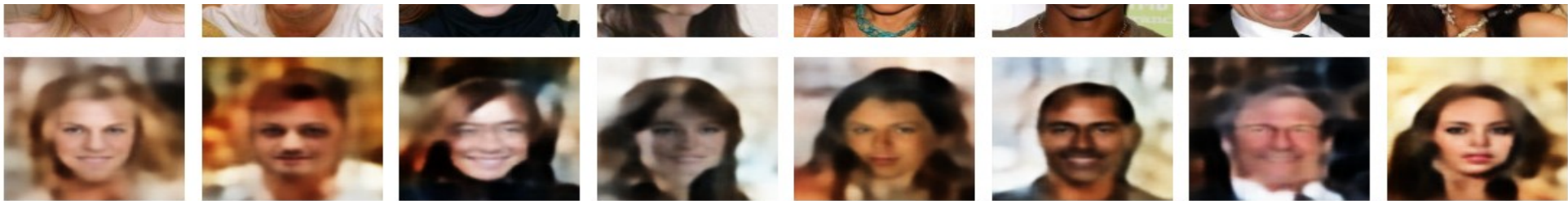
Shows images from the dataset

Shows the same images after being passed through the AE

Reconstructing images:



Adding noise in the latent space:



Decoding random points in the latent space:



5B.9 Using VAEs to generate new faces

Encoder: has three additional elements:

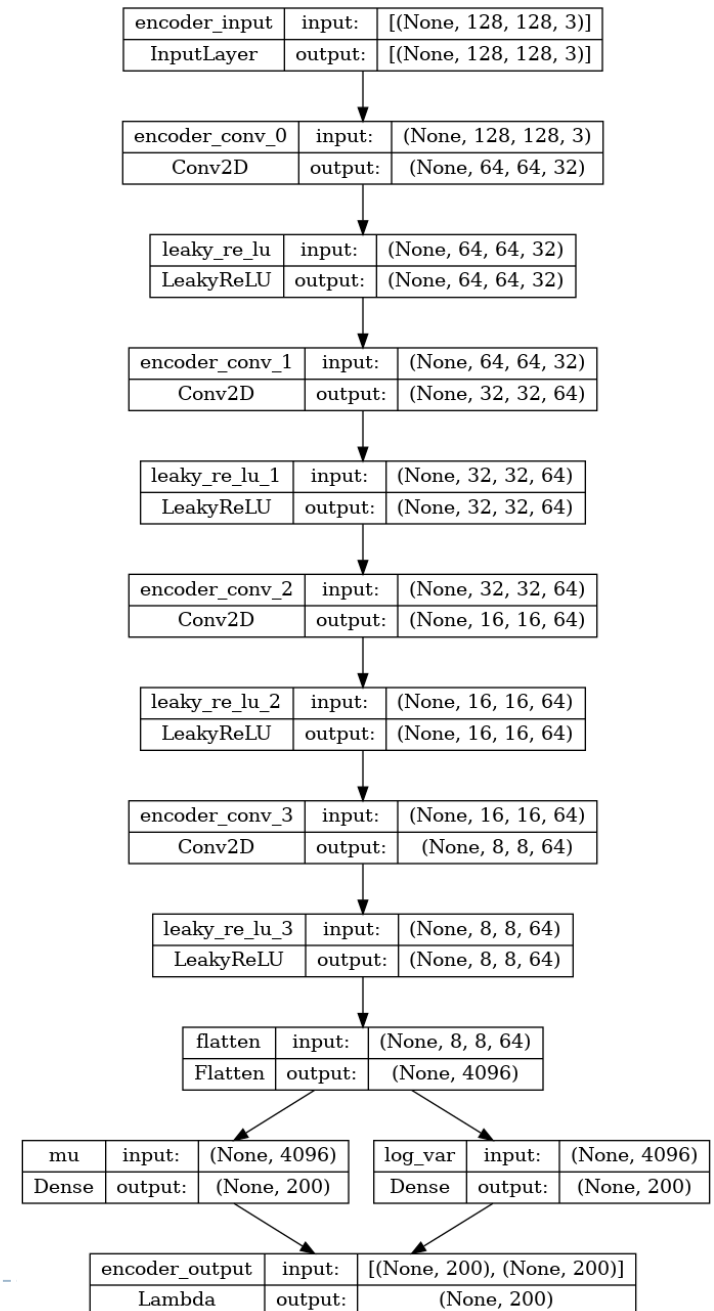
- ▶ μ : the mean μ
- ▶ $\log_var$: the logarithm of the variance σ
- ▶ A sampling function

Decoder: basically a mirror image of the encoder

Loss function is the sum of **RMSE** and **KL divergence**.

A weight (the loss factor) is assigned to the RMSE loss:

- ▶ High loss factor $\Rightarrow$ VAE would be similar to a simple AE
- ▶ Low loss factor $\Rightarrow$ the quality of the reconstructed images will be poor



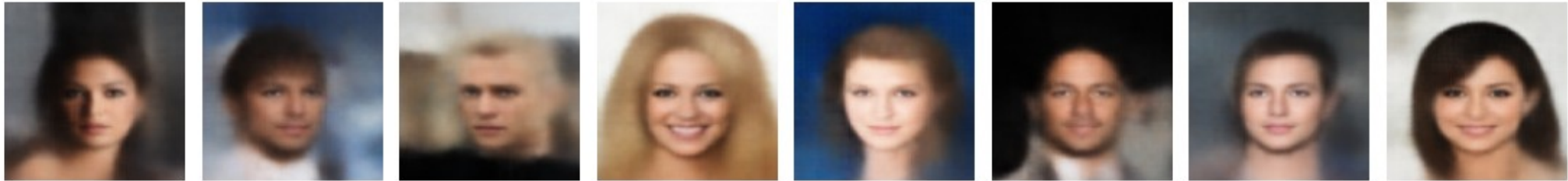
Original



AE



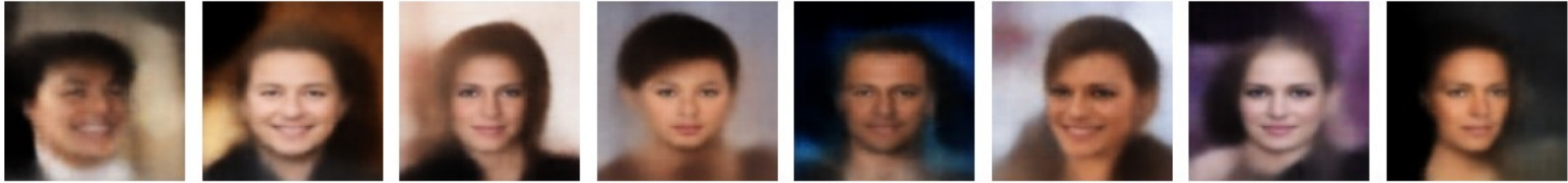
VAE



Adding noise in the latent space:

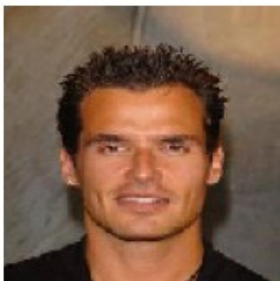


Creating faces from random points in the latent space:



Interpolating from two faces in the latent space:





Introducing perturbations in
different dimensions in the
latent space

